instituto superior de engenharia de lisboa

Fundamentos de Sistemas Operativos

2º. Trabalho Prático

Mª Luísa Melo e Sampaio
No. Aluno 50799

Carlos Ceita
No. Aluno 50950

Grupo 08

Prof. Eng.º Jorge Pais
Docente Responsável

Instituto Superior de Engenharia de Lisboa

Departamento de Engenharia Informática
Licenciatura em Engenharia Informática e Multimédia

Lisboa, 22 de Dezembro de 2025

Conteúdo

Lista de Figuras iii

Lista de Tabelas iv

Lista de Códigos v

Glossário vi

Siglas vii

1 Introdução **1**

1.1 Objetivos 2

2 Tarefa Evitar Obstáculo **3**

2.1 Objetivos da Aula 3

2.2 Tarefa Evitar Obstáculo 3

2.2.1 Desenvolvimento da Tarefa 3

2.3 Resultados Práticos 5

3 GUI Gravador **6**

3.1 Objetivos da Aula 6

3.2 Tarefa Gravador 6

3.3 GUI do Gravador 6

3.4 Resultados Práticos 8

4 Tarefa Gravador **9**

4.1 Objetivos da Aula 9

4.2 Desenvolvimento da Tarefa 9

4.2.1 Função de Gravar 10

4.2.2 Função Reproduzir 11

4.2.3 Métodos Importantes da Classe 13

4.3 Swing não é Thread-Safe 15

4.4 Resultados Práticos 15

5 Sincronização EvitarObstáculos e Gravador **16**

5.1 Objetivos da Aula 16

5.2 Explicação do Problema 16

5.3 Resolução do Problema 17

5.4 Resultados Práticos 18

6 Sincronização de Robots **19**
6.1 Objetivos da Aula 19
6.2 Implementação 19
6.3 Resultados Práticos 20

7 Polimorfismo e Herança **21**
7.1 Herança 21
7.1.1 Importância da Herança 21
7.2 Polimorfismo 21
7.2.1 Importância do Polimorfismo 21
7.3 Diagrama de Classes Completo 22

8 Conclusões **23**

Bibliografia 24

Código **25**

Lista de Figuras

2.1 Diagramas da tarefa `EvitarObstaculos` 4
2.2 Interface Gráfica com botão Evitar Obstáculos 5

3.1 Diagrama da Classe GUI_Gravador 7
3.2 Interfaces Gráficas Tarefa Gravador 7
3.3 Processo de escolha de ficheiro 8

4.1 Diagrama de Classe `Gravador` 9
4.2 Diagrama de Estados Gravar 10
4.3 Processo de gravação num ficheiro 11
4.4 Diagrama de Estados Reproduzir 11
4.5 GUI Reproduzir 12
4.6 Processo de Gravação e Reprodução de Movimentos Aleatórios 13

5.1 Diagrama de Atividades Tarefas EvitarObstáculos e MovAleatorios (Versão Simplificada) 17

7.1 Diagrama de Classes Completo 22

Lista de Tabelas

4.1 Conversão Comando → String 14
4.2 Conversão String → Comando 14

Lista de Códigos

6.1 Método de Seleção do Robot 19
6.2 Método isPartilhado() 20
1 Código Classe App 26
2 Código Classe BufferCircularComando 26
3 Código Classe Comando 28
4 Código Classe CRobotLegoEV3 30
5 Código Classe Dados 31
6 Código Classe EvitarObstaculos 34
7 Código Classe Gravador 36
8 Código Classe GUI 41
9 Classe GUIGravador 50
10 Código Interface IRobotLegoEV3 61
11 Código Classe MovAleatorios 61
12 Código Classe MyRobotLegoEV3 64
13 Código Classe MyRobotSync 68
14 Código Classe RobotSimulado 69
15 Código Classe Abstrata Tarefa 70
16 Código Classe Auxiliar TempoMovimento 72

Glossário

Java Swing Java Swing é um conjunto de bibliotecas de classes Java para criar interfaces gráficas de utilizador GUI de forma visual e interativa. (*p. 1*)

WindowBuilder O WindowBuilder é um plugin para o Eclipse que permite criar interfaces gráficas em Java através de um designer visual, eliminando a necessidade de escrever código manual. (*p. 1*)

Siglas

EDT Event Dispatch Thread. (*p. 15*)

FSO Fundamentos de Sistemas Operativos. (*p. 1, 2*)

GUI *Graphic User Interface* - Interface Gráfica. (*p. i, iii, vi, 1, 4, 6, 8, 12, 13*)

TP2 Trabalho Prático 2. (*p. 19*)

UC Unidade Curricular. (*p. 1*)

Capítulo 1

Introdução

A Unidade Curricular de **Fundamentos de Sistemas Operativos** (**FSO**) avalia os conhecimentos adquiridos pelos alunos por meio de duas componentes de igual peso: *i*) a componente teórica, correspondente a um exame escrito; *ii*) e a componente laboratorial, que consiste em parte no desenvolvimento e documentação de dois projetos de grupo – entre os quais se inclui o presente documento.

O presente relatório documenta o desenvolvimento de uma aplicação Java para o controlo de um robot Lego EV3. O projeto tem como objetivo principal a aplicação prática de conhecimentos fundamentais de programação concorrente e sincronização de tarefas.

O sistema desenvolvido consiste numa aplicação multitarefa composta por uma *Graphic User Interface* - Interface Gráfica (GUI) e por um conjunto de tarefas autónomas que operam sobre o robot. O projeto foi estruturado em duas fases de desenvolvimento distintas.

Numa primeira fase, o grupo focou-se na criação da infraestrutura base de comunicação com o robot e no desenho da GUI usando a biblioteca Java Swing e a ferramenta WindowBuilder. Nesta etapa, foi implementa a tarefa `Movimentos Aleatórios`, que gera e envia automaticamente sequências de comandos para o robot, e estabeleceu-se o padrão produtor-consumitor para gerir o fluxo de dados entre a interface e o robot.

Na segunda fase, o sistema foi expandido com duas novas funcionalidades: a tarefa `EvitarObstáculos` e `Gravador`. A primeira usa o sensor de toque do robot para detetar colisões e executar manobras para voltar atrás. A segunda adicionou a capacidade de gravar uma sequência de comandos num ficheiro texto para depois ser reproduzido.

Um dos maiores desafios encontrados em ambas as fases foi a gestão do acesso aos recursos partilhados. Para assegurar a integridade do funcionamento do robot e evitar conflitos entre as múltiplas tarefas, foram implmentados mecanismos de sincronização usando semáforos e monitores.

1.1 Objetivos

> 2º Trabalho Prático - `TrabalhoPratico2.pdf` - (Moodle de Fundamentos de Sistemas Operativos 2025/2026)

Os principais objetivos do projeto são:

1. **Desenvolvimento de interfaces gráficas** em Java Swing, usando a ferramenta WindowBuilder para o Eclipse.
2. **Implementação de uma aplicação multitarefa** capaz de executar várias *threads* em simltâneo.
3. **Aplicação de mecanismos de sincronização** entre tarefas, especificamente semáforos e monitores para resolver problemas de acesso concorrente ao robot e garantir a segurança das operações.
4. **Manipulação de ficheiros** recorrendo às classes `InputStream`, `Reader`, `OutputStream` e `Writer` do Java.

Capítulo 2

Tarefa Evitar Obstáculo

2.1 Objetivos da Aula

Na primeira aula o grupo foi desafiado a. desenvolver e testar a tarefa "Evitar Obstáculo". Assim como a integração da tarefa no diagrama de classes desenhado no MagicDraw.

2.2 Tarefa Evitar Obstáculo

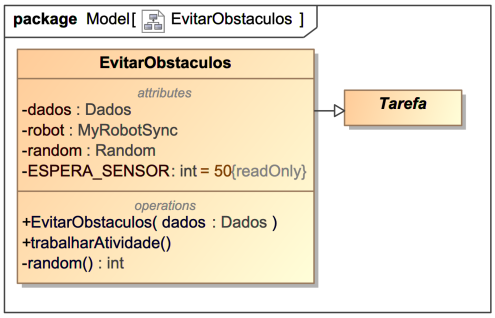
Esta tarefa testa o sensor de toque de 50 em 50 ms, e quando este ficar ativo, deve **imediatamente** parar o robot. De seguida executa uma série de comandos:

- Recua 20cm.
- Curva à esquerda ou direita (com escolha aleatória) de 90 graus.

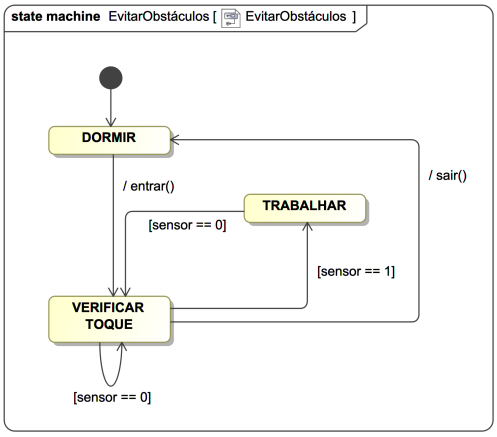
Durante estas ações, nem o utilizador nem as outras tarefas podem movimentar o robot.

2.2.1 Desenvolvimento da Tarefa

Para esta nova tarefa, foi criada uma nova classe `EvitarObstaculos`. Esta classe vai estender de `Tarefa`.



(**a**) *Diagrama da classe* `EvitarObstaculos`



(**b**) *Diagrama de estados* `EvitarObstaculos`

Figura 2.1: *Diagramas da tarefa* `EvitarObstaculos`

A implementação da tarefa foca-se no método `trabalharAtividade()`. Este método implementa um ciclo que a cada 50 milissegundos (`Thread.sleep(50)`), verifica o sensor de toque (`robot.SensorToque()`).

O digrama de estados 2.1b mostra o funcionamento desta tarefa. Ao detetar uma colisão (valor `1`), a tarefa invoca logo o método para parar o robot `dados.getRobot().Parar(true)`. Imediatamente a seguir, adquire o controlo exclusivo do robot através da chamada ao semáforo `dados.getRobot().entrar()`. De seguida, vai limpar o buffer para dar prioridade à tarefa. Com o acesso garantido, executa a manobra de evasão: primeiro recua 20 cm (`Reta(-20)`) e depois executa aleatoriamente uma curva de 90 graus para a esquerda ou direita (`CurvarEsquerda(0, 90)` ou `CurvarDireita(0, 90)`).

De forma crucial, a chamada para libertar o semáforo, `dados.getRobot().sair()`, está num bloco `finally`. Isto garante que o robot não fica bloqueado (evitando *deadlock*) mesmo que ocorra um erro durante a manobra. A ativação e desativação desta tarefa são controladas pelo utilizador através do botão "Evitar Obstáculos"na GUI.

No final, se a tarefa de Movimentos Aleatórios continuar ativa, o robot volta a esse estado.



Figura 2.2: *Interface Gráfica com botão Evitar Obstáculos*

2.3 Resultados Práticos

No final desta aula, o grupo foi capaz de desenvolver por completo a tarefa Evitar Obstáculos. A implementação cumpriu os requisitos funcionais e técnicos definidos para esta etapa do projeto.

Foi implementada a lógica central no método `trabalharAtividade()`, que verifica o sensor de toque a cada 50 milissegundos. Ao detetar uma colisão, a tarefa executa as manobras definidas.

Capítulo 3

GUI Gravador

3.1 Objetivos da Aula

Na segunda aula prática, foi desenvolvida a interface para a tarefa **Gravador** assim como o tratamento de eventos dos componentes gráficos.

3.2 Tarefa Gravador

Esta tarefa permite gravar ou reproduzir num ficheiro à escolha do utilizador, um conjunto de comandos, quer sejam enviados diretamente pelo utilizador ou dos movimentos aleatório.

Esta tarefa tem a possibilidade de trabalhar com o mesmo robot ou com um robot diferente das outras tarefas. A manipulação do foi feita com a ajuda das classes de `InputStream`, `Reader`, `OutputStream` e `Writer` do `java.io`.

3.3 GUI do Gravador

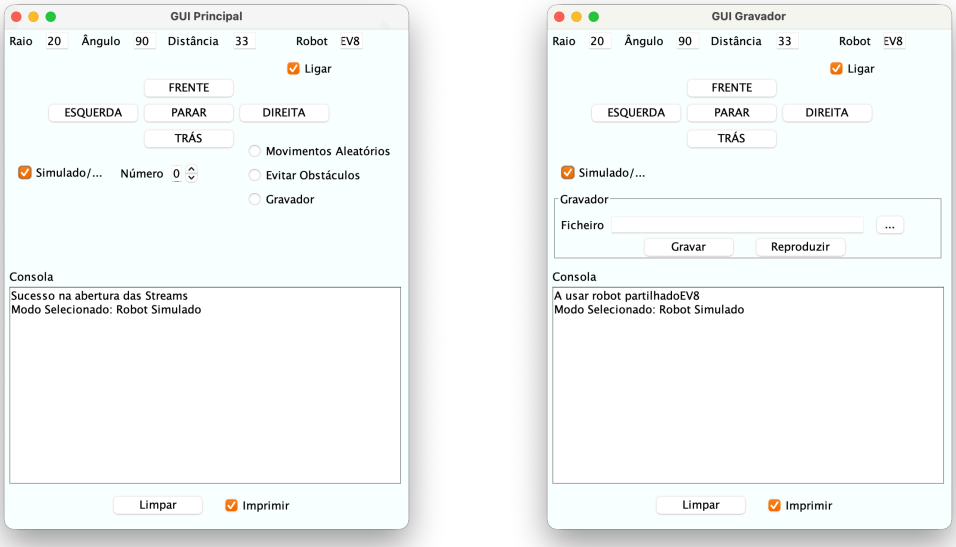
A GUI desta tarefa é dividida em 3 campos:

- Um campo que permite interagir com o robot.
- Outro que define o ficheiro para a gravação ou reprodução e qual a funcionalidade.
- O último que define a consola tendo as opções de limpeza e de permissão para imprimir.



Figura 3.1: *Diagrama da Classe GUI_Gravador*

Como se pode ver na figura 3.1, esta classe é muito semelhante a classe da GUI Principal. Na figura Porém esta foi adaptada para a tarefa do Gravador.



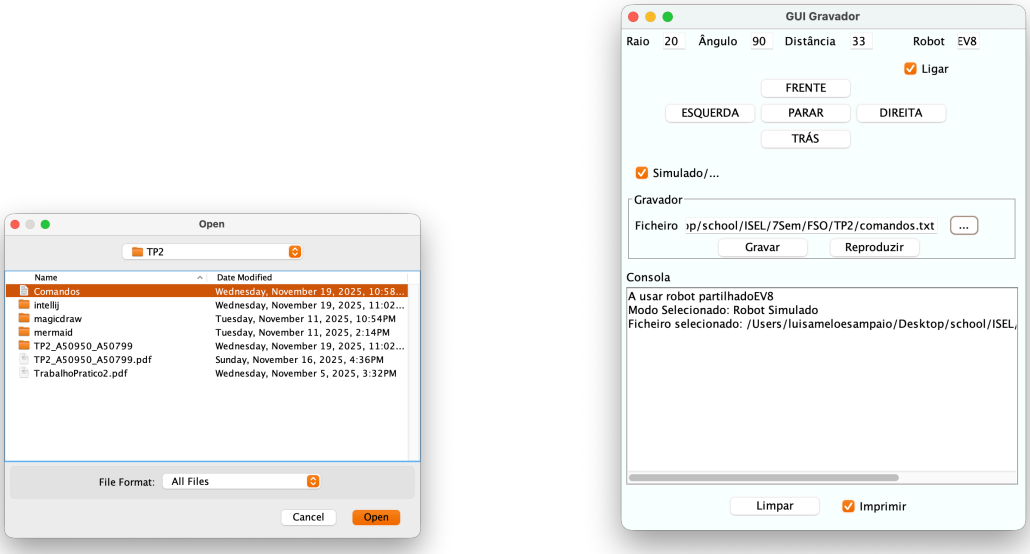
(**a**) *Interface Gráfica Principal* (**b**) *Interface Gráfica* `Gravador`

Figura 3.2: *Interfaces Gráficas Tarefa Gravador*

Quando o utilizador quiser utilizar a funcionalidade do Gravador, seleciona a opção

na GUI principal (Figura 3.2a) que vai iniciar a GUI do Gravador (Figura 3.2b). Pode se escolher utilizar o mesmo robot ou utilizar um robot diferente, pondo o seu nome no `JTextField` "Robot".

Ao clicar no botão de escolha de ficheiros, abre-se uma janela onde é possivel escolher o ficheiro onde gravar ou para reproduzir os movimentos.



(**a**) *Escolha do Ficheiro* (**b**) *Ficheiro na GUI*

Figura 3.3: *Processo de escolha de ficheiro*

3.4 Resultados Práticos

Nesta segunda aula, o principal objetivo foi concretizada com o desenvolvimento e integração da GUI.

Foi possível consolidar os conhecimentos sobre a manipulação de ficheiros em Java. Esta funcionalidade é importante para o projeto, pois permite a gravação e reprodução de comandos do robot.

Capítulo 4

Tarefa Gravador

4.1 Objetivos da Aula

Para a terceira a aula, o grupo teve como objetivo o teste e a implementação da tarefa **Gravador**.

4.2 Desenvolvimento da Tarefa

Tal como foi mencionado na Secção 3.2, esta tarefa pode gravar comandos num ficheiro para posteriormente reproduzi-los.

Para o seu desenvolvimento, foi criada a classe `Gravador`(8) que vai estender de `Tarefa`.

Depois de uma análise dos slides do docente [1, p.38-39] e da ideia que o grupo tinha para a implementação desta classe, foram escolhidas as classes `java.io.BufferedReader`, `java.io.Writer` `FileOutputStream`, `OutputStreamWriter`, `java.io.InputStream`, `java.io.InputStreamReader`, `java.io.Reader` e `java.io.OutputStream`

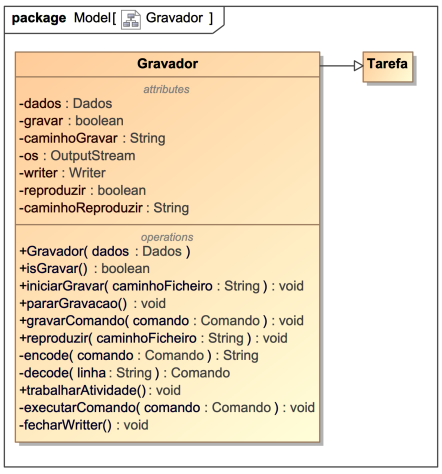


Figura 4.1: *Diagrama de Classe* `Gravador`

As duas funcionalidades principais são:

1. Gravar comandos num ficheiro com a ajuda das classes `OutputStream` e `Writer`.
2. Reproduzir um ficheiro de comandos com `InputStream` e `Reader` e executar esses comandos no robot em exclusão mútua.

4.2.1 Função de Gravar

Esta função permite ao utilizador ou as outras tarefas (EvitarObstáculos e Movimentos Aleatórios), guardar uma sequência de comandos enviados ao robot. O utilizador começa por escolher o ficheiro texto onde quer gravar os comandos. Assim que clicar no botão de gravar, todo movimento enviado para o robot será gravado.

Sempre que um comando for enviado, a GUI do Gravador chama o método `dados.registarComandos(comando)` que por si mesmo chama o método da classe `Gravador`, `gravarComando(comando)` para escrever uma linha no ficheiro.

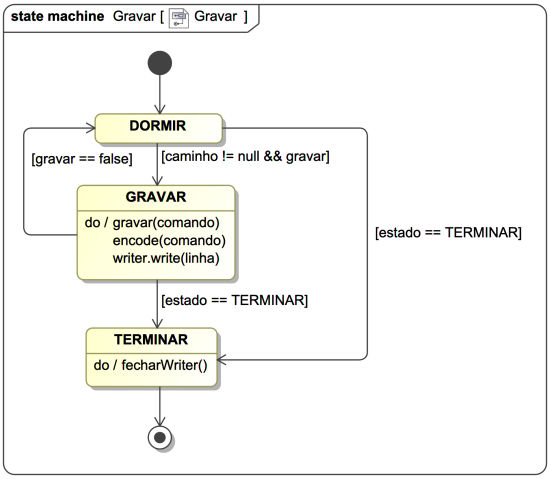
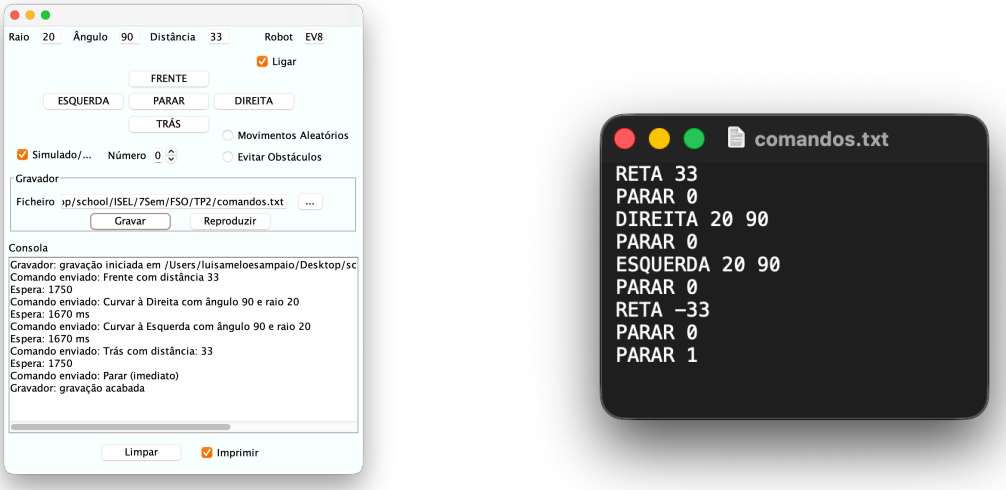


Figura 4.2: *Diagrama de Estados Gravar*

O modo de gravação não é implementado como uma máquina de estados concorrente, uma vez que não é preciso de execução autónoma nem de sincronização ativa. O seu funcionamento é baseado num modo ativo/inativo controlado por uma variável *boolean*, sendo os eventos de gravação desencadeados externamente pelas ações do utilizador e pelos comandos do robot.

Formato do ficheiro

O ficheiro vai ficar com o seguinte formato:



(**a**) *Comandos na Consola* (**b**) *Comandos no Ficheiro*

Figura 4.3: *Processo de gravação num ficheiro*

Como podemos ver na figura 4.6b temos multiplos "Parar":

- **PARAR 0**: Um "PARAR"mandado naturalmente a seguir a cada comando.
- **PARAR 1**: Um "PARAR"mandado pelo o utilizador que na consola aparece como `Comando Enviado: Parar(imediato)` pois este tem prioridade sobre todas as tarefas e para imediatamente o robot.

4.2.2 Função Reproduzir

Esta função permite ao utilizador de escolher um ficheiro que contenha comandos, para que possam ser feitos pelo robot.

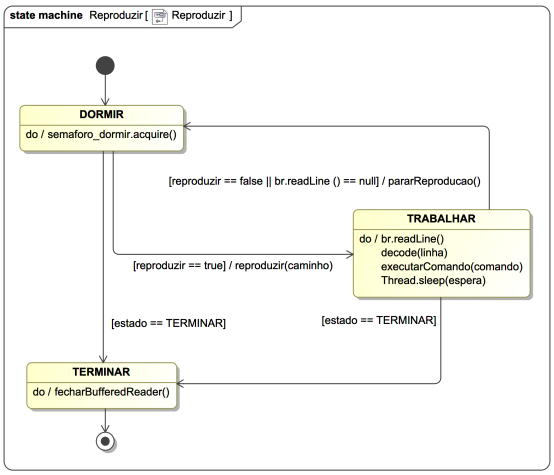


Figura 4.4: *Diagrama de Estados Reproduzir*

A figura 4.4 mostra o diagrama de estados da funcionalidade reproduzir. Esta funcionalidade, ao contrário da Gravação, já foi implementada como uma máquina de estados pois exige esclusão mútua. A tarefa inicia-se no estado **DORMIR**, onde a thread permanece bloqueada por um semáforo.

Quando o método `reproduzir()` é invocado com um caminho válido, a tarefa é ativada, passando para o estado **TRABALHAR**. Nesta fase inicial, são verificadas as condições necessárias à execução, nomeadamente se o robot se encontra ligado. Caso esta condição não seja satisfeita, a reprodução é cancelada e a tarefa regressa ao estado **DORMIR**.

Quando o robot estiver ligado, a tarefa começa a reprodução, onde é efetuada a leitura sequencial do ficheiro de comandos. Cada linha é validada, descodificada e executada no robot, podendo existir períodos de espera associados à execução dos comandos. A tarefa permanece neste estado enquanto existirem comandos a executar e não ocorrer qualquer interrupção.

A reprodução pode terminar de 3 formas: conclusão do ficheiro, interrupção externa pelo utilizador ou ocorrência de um erro. Em qualquer um dos casos, o robot é parado de forma controlada e volta ao estado **DORMIR**, garantindo um retorno a um estado seguro e consistente.

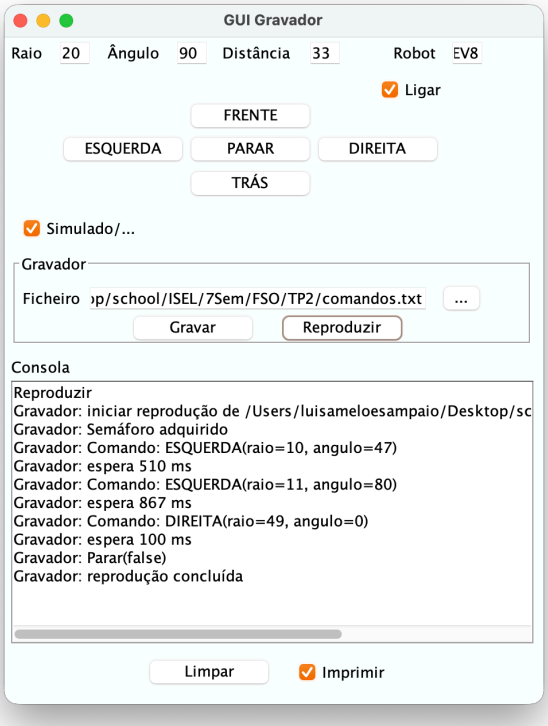
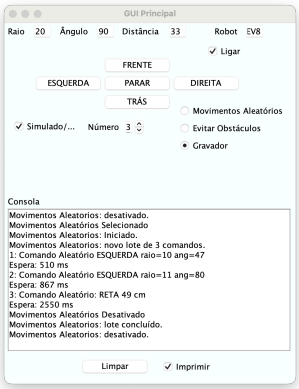


Figura 4.5: *GUI Reproduzir*

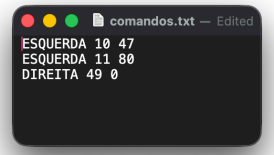
Podemos ver na figura 4.5 que o robot reproduziu corretamente os comandos gravados no ficheiro.

Gravar Movimentos Aleatórios

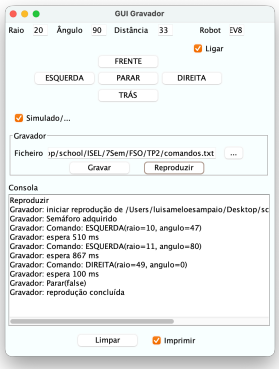
Tal como é feito na GUI, cada vez que um comando for criado na classe de movimentos aletórios, também vai ser chamada a função `dados.registarComandos()` com o comando correspondente. Resulta então nas seguintes figuras:



(**a**) *Comandos Movimentos Aleatórios na Consola*



(**b**) *Comandos Movimentos Aleatórios no Ficheiro*



(**c**) *GUI Reprodução Movimentos Aleatórios*

Figura 4.6: *Processo de Gravação e Reprodução de Movimentos Aleatórios*

Como podemos ver na figura 4.6, os comandos dos movimentos aleatórios foram corretamente gravados no ficheiro e reproduzidos.

Sincronização

A escrita no ficheiro foi desenvolvida com monitores pois o grupo achou uma tarefa mais simples e que não seria necessária a complexidade dos semáforos. Assim, fechar o *Writer* e o ato de escrever não acontecem ao mesmo tempo.

Porém, para a reprodução, a tarefa obteve o semáforo no ínicio e liberou no final pois ninguém pode manipular o robot quando este está a repoduzir um ficheiro de comandos.

4.2.3 Métodos Importantes da Classe

`iniciarGravar()`

Este método foi desenvolvido usando como base o seguinte exemplo: [2] Começa primeiro por fazer umas verificações, e só depois é que vai abrir um `FileOutputStream` e um `OutputStreamWriter`. Por fim, troca a *flag* de gravar para verdadeira.

`pararGravacao()`

Este método fecha o `Writer` e o `FileOutputStream` com o método auxiliar `fecharWriter()` e define a *flag* como falsa.

gravarComando(Comando comando)

Este é o método principal para gravar os comandos no ficheiro. É chamado na classe `Dados` quando for preciso gravar o comando. Escreve a linha usando o método `encode()` e acrescenta uma quebra de linha com `"\n"` para facilitar a leitura do ficheiro.

encode(Comando comando)

Esta função obtém o identificador do comando enviado e retorna em String a linha para ser gravada no ficheiro:

Descrição	Identificador	Linha no Ficheiro
RETA	0	Reta <distância>
CURVAR_DIR	1	CurvaDireita <raio> <ângulo>
CURVAR_ESQ	2	CurvaEsquerda <raio> <ângulo>
PARAR	3	Parar <escolha>

Tabela 4.1: *Conversão Comando → String*

decode(String linha)

Ao contrário da função `encode()`, esta recebe uma linha e com a ajuda de `\\s+` vai separar as palavras a cada espaço e pô-las em cada indice de um array [3]. De seguida vai analisar o indíce 0 do array e fazer uma verificação para depois mandar retornar o comando correto.

Linha no Ficheiro	Comando Convertido
RETA 10	["RETA", "10"]
DIREITA 45 90	["DIREITA", "45" , "90"]
ESQUERDA 15 20	["ESQUERDA", "15", "20"]
PARAR 1	["PARAR", "1"]

Tabela 4.2: *Conversão String → Comando*

trabalharAtividade()

Método da classe abstrata `Tarefa`. O seu principal objetivo é garantir que ninguém tem acesso ao robot durante a reprodução de comandos. Para tal, adquire o semáforo assim que a funcionalidade de reprodução for ativa. Vai usar as classes `InputStream`, `FileInputStream`, `Reader InputStreamReader` e `BufferedReader` [4]. No final da reprodução, volta a liberar o semáforo.

esperarComando(Comando comando)

Este método vai ser chamado dentro do `trabalharAtividade()` fazer a espera necessária para cada comando.

4.3 Swing não é Thread-Safe

O Swing não é "**thread-safe**", significa que os componentes de interface gráfica do *Swing* não podem ser criados, acedidos ou modificados com segurança por várias threads ao mesmo tempo.

No Java, o *Swing* tem uma **thread especial** chamada Event Dispatch Thread (EDT).

Essa *thread* é responsável por gerir todos os eventos da interface gráfica, como cliques de botão, eventos do teclado e modificações visuais dos componentes.

Quando se modifica uma interface gráfica, por exemplo, mudar o texto de uma JLabel, modificar JTextField, adicionar um JButton a um JPanel, etc, tudo deve acontecer dentro da EDT. [5]

Se uma *thread* da aplicação tentar modificar os componentes *Swing* pode ocorrer:

1. Comportamento imprevísivel,
2. Bloqueio de *thread* (*deadlock*),
3. Erros visuais (componentes não atualizam corretamente),
4. Geração de exceções.

Para tal, a classe `App` foi mudada para implementar a interface `Runnable`.

4.4 Resultados Práticos

A realização desta aula permitiu ao grupo atingir com sucesso o objetivo de implementar a persistência de dados no controlo do robot, através da tarefa **Gravador**.

Foi possivel aprofundar os conhecimentos sobre sincronização e concorrência. O grupo optou-se pela simplicidade dos **monitores** para garantir a exclusão mútua na escrita do ficheiro, enquanto se utilizou o **semáforo** existente para a reprodução, garantindo que o robot opera em exclusividade absoluta durante a execução de uma sequência gravada.

Em suma, a implementação da classe `Gravador` não só adicionou uma funcionalidade vital de reprodução ao sistema, como também consolidou a aplicação prática de mecanismos de exclusão mútua e boas práticas de desenvolvimento de interfaces gráficas em Java.

Capítulo 5

Sincronização entre EvitarObstáculos e Gravador

5.1 Objetivos da Aula

Esta aula focou-se numa melhor sincronização entre as 3 tarefas disponiveis. O grupo deparou-se com um erro de conflito de acesso às diferentes tarefas. Este conflito lançava o seguinte erro para o terminal:

`Exclusão mútua desrespeitada no método Reta(...)!`[1]

5.2 Explicação do Problema

A biblioteca `RobotLegoEV3` espera que **apenas uma tarefa** aceda ao robot de cada vez. Quando o `EvitarObstáculos` chama o `SensorToque()` a cada 50ms **enquanto** outras tarefas estão a enviar comandos, a biblioteca lança o erro.

Apesar do código continuar a funcionar, estes erros podem ser comprometedores. Estava a acontecer o seguinte:

1. **Tarefa EvitarObstáculos (Sensor):** Estava a adquirir corretamente o semáforo antes de ler o sensor.
2. **Tarefa MyRobotLegoEV3 (Consumidor):** Esta é a tarefa interna que lê os comandos do buffer circular e envia-os para o robot. Esta thread não usava semáforo.

Ora, quando a tarefa do **Consumidor** estava a enviar um comando de movimento (`Reta(distância)`, `CurvarDireita(raio, ângulo)`...), e ao mesmo tempo, a tarefa de **Evitar Obstáculos** tenta ler o sensor, acontece a colisão na biblioteca, gerando o erro.

[1] Este é apenas um exemplo representativo. O erro de exclusão mútua acontecia em quase todos os métodos da biblioteca quando solicitados ao mesmo tempo.

5.3 Resolução do Problema

Para resolver este conflito, foi preciso garantir que a tarefa do consumidor partilha o mesmo semáforo do que na classe `MyRobotSync`. Ao fazer `semaforo.acquire()` dentro do `run()` do `MyRobotLegoEV3`, garante-se que o comando só é enviado para o robot quando mais ninguém está a usar o robot. O `semaforo.release()` é feito logo a seguir ao envio do comando. Assim, a tarefa `EvitarObstáculos` consegue adquirir o semáforo para ler o sensor de toque.

Porém, ao fazer estas mudanças, o robot deixou de reproduzir os comandos. Estes eram lidos e impressos na consola, mas não eram executados no mundo físico.

Estava a acontecer um bloqueio no `trabalharAtividade()` da classe `Gravador`.

Tentava-se adquirir o semáforo antes de começar o ciclo de leitura do ficheiro para que o tivesse durante toda a reprodução. De seguida, a classe envia o comando para o buffer e a *thread* consumidora, que modificámos anteriormente retira o comando do buffer e tenta fazer `semaforo.acquire()` para poder comunicar com o robot.

No bloqueio, o robot ficava eternamente à espera que o semáforo porque o `Gravador` nuna o chegou a libertar (só o libertaria no fim do ficheiro). Como o robot não executa o comando, o `Gravador` (que espera pelo tempo de movimento) e o robot ficam ambos parados.

Para tal, foram removidas as tentativas de acesso ao semáforo no `Gravador`. O controlo de acesso ao robot acabou por ser feito comando a comando pelo próprio robot.

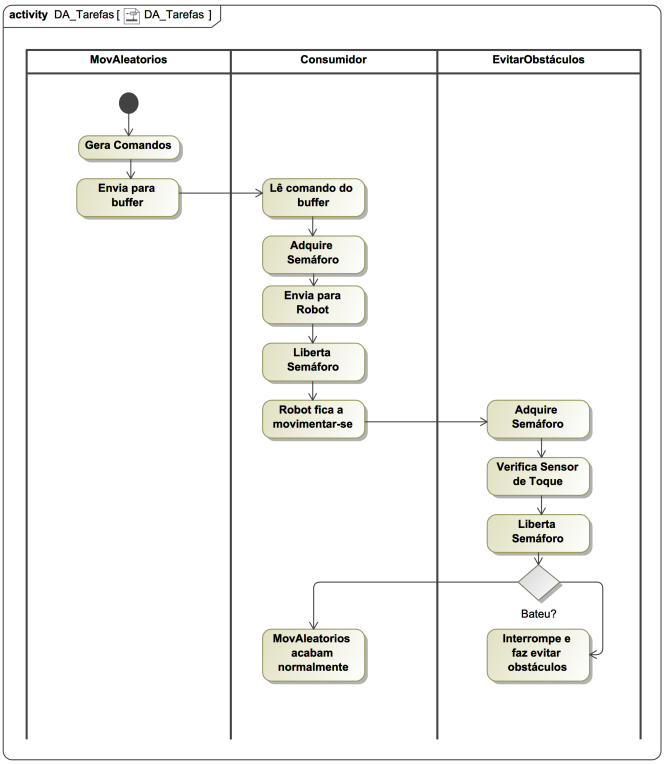


Figura 5.1: *Diagrama de Atividades Tarefas EvitarObstáculos e MovAleatorios (Versão Simplificada)*

5.4 Resultados Práticos

Nesta aula o grupo foi capaz de introduzir uma sincronização mais eficaz e segura.

Foi possivel aprofundar ainda mais os conhecimentos de sincronização e conflitos. Com poucas mudanças, o robot ficou a funcionar normalmente.

Ainda assim, o Problema que se teve da reprodução estar a funcionar antigamente e depois das mudanças também ter que se adaptar a parte do código responsável, fez o grupo perceber ainda mais a importância de uma correta programação por objetos.

Capítulo 6

Sincronização de Robots

6.1 Objetivos da Aula

Na ultima aula prática dedicada ao TP2, o principal objetivo foi adaptar a aplicação para ter mais flexibilidade no controlo dos robots permitindo dois modos de operação distintos:

1. **Controlo Partilhado**: Ambas as interfaces (GUI Principal e GUI Gravador), controlam o mesmo robot, garantindo a sincronização dos comandos.
2. **Controlo Independentes**: A GUI do Gravador controla um segundo robot, permitindo a execução de tarefas paralelas.

6.2 Implementação

Para concretizar estes objetivos, implementou-se na classe `GUI_Gravador` um mecanismo de gestão dinâmica da instância do robot.

A lógica desta decisão foi feita no método `getRobotAtual()`, como se pode observar na listagem abaixo:

```
1 public MyRobotSync getRobotAtual() {
2         if (nomeAtual.equals(dados.getNomeRobot()) && dados.isOnOff()) {
3             return dados.getRobot();
4         }
5         return robotLocal;
6     }
```

Listing 6.1: *Método de Seleção do Robot*

O sistema vai verificar o nome do robot inserido pelo utilizador na caixa de texto da `GUI_Gravador` e compara-o com o nome do robot ativo na classe `Dados`.

- **Modo Partilhado**: Se os nomes forem iguais, o método retorna a referência ao objeto partilhado. Neste caso, a exclusão mútua é assegurada pelo Semáforo da classe `MyRobotSync`, impedindo que comandos de janelas diferentes colidam.

- **Modo Independente**: Se os nomes forem diferentes, a aplicação instancia um novo objeto do tipo `MyRobotSync` e estabelece a conexão dedicada. Isto permite controlar dois robots diferentes sem quaisqueres problemas.

Para ajudar nos diferentes componentes da classe `GUI_Gravar`, foi criado o método `isPartilhado` que simplesmente retorna um *boolean* caso a String (nome do robot) posta no campo de texto seja igual à que está na GUI Principal.

```
1 private boolean isPartilhado(){
2         return nomeAtual.equals(dados.getNomeRobot());
3     }
```

Listing 6.2: *Método isPartilhado()*

Utiliza-se este método em 3 momentos importantes:

1. **Mudança de Robot**: Quando o utilizador muda o nome na caixa de texto, vai se invocar este método para saber se deve, ou não, fechar a conexão do robot local antigo ou se deve deixar de referenciar o robot partilhado.
2. **Botão Ligar/Desligar**: Ao clicar em "Ligar"o método determina se é necessário abrir uma nova conexão ao apenas validar se o robot principal já está ligado.
3. **Prevenção de Erros**: Garante que a aplicação não tenta fechar a conexão do robot principal a partir da janela secundária, protegendo a integridade da sessão da GUI Principal.

6.3 Resultados Práticos

No final desta aula, o grupo conseguiu cumprimr os objetivos propostos para permitir que o utilizador escolah livremente entre operar um único robot de forma colaborativa ou gerir vários robots simulataneamente.

Capítulo 7

Polimorfismo e Herança

7.1 Herança

A herança é um mecanismo da Programação Orientada a Objetos que permite criar novas classes a partir de classes existentes, reutilizando e estendendo os seus atributos e métodos. Estabelece uma relação "é um"(is-a) entre a classe filha (subclasse) e a classe mãe (superclasse). [1]

Neste projeto, a herança é usada centralmente na classe abstrata **Tarefa** que estende da classe **Thread**.

As classes `MovAleatorios`, `EvitarObstaculos` e `Gravador` herdam de `Tarefa`.

7.1.1 Importância da Herança

Isto permite a reutilização de código, centralizando a gestão da máquina de estados (DORMIR, TRABALHAR, TERMINAR) e o controlo do semáforo de ativação na classe mãe, evitando a repetição de lógica em cada tarefa nova. Também foi usada na GUI, que herda de JFrame para obter as funcionalidades de uma janela.

7.2 Polimorfismo

O polimorfismo permite que objetos de diferentes classes sejam tratados como objetos de uma superclasse ou interface comun. O método específico executado é determinado em tempo de execução, dependendo do tipo real do objeto.

Neste projeto, o polimorfismo é fundamental na abstração do hardware do robot através da interface `IRobotLegoEV3`.

As classes `CRobotLegoEV3`, `RobotSimulado` e `MyRobotSync` implementam esta interface.

7.2.1 Importância do Polimorfismo

O polimorfismo permite que a aplicação envie comandos como `Reta()` ou `Parar()` sem saber se está a controlar o robot real ou o simulado. Isto confere flexibilidade

ao sistema, permitindo trocar o modo de funcionamento em tempo de execução sem alterar o código.

7.3 Diagrama de Classes Completo

A figura abaixo demontra o diagrama de classes completo, onde é possível ver os diferentes casos de **Herança** e **Polimorfismo**

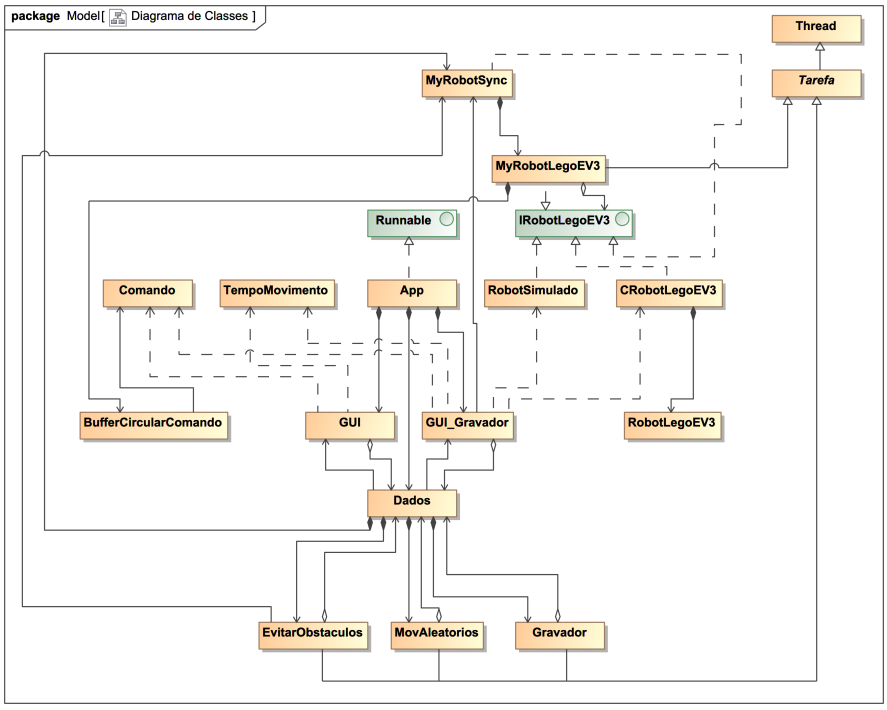


Figura 7.1: *Diagrama de Classes Completo*

Capítulo 8

Conclusões

O desenvolvimento deste segundo trabalho prático permitiu a aplicação de conceitos fundamentais de programação concorrente e sincronização no contexto de controlo de um robot Lego EV3. Ao longo do projeto, a implementação de uma aplicação multitarefa demonstrou ser um desafio técnico, particularmente na gestão do acesso exclusivo aos recursos partilhados para evitar conflitos de exclusão mútua. Através da utilização de semáforos e monitores, foi possível garantir que tarefas autónomas como a de "Evitar Obstáculos"e a de "Movimentos Aleatórios"operassem de forma coordenada.

Os objetivos principais definidos para o projeto foram concluídos com sucesso, permitindo o controlo eficaz do robot, a reprodução de comandos em ficheiro e a gestão de múltiplas threads em simultâneo. No entanto, reconhece-se que a solução final poderá não apresentar a implementação mais otimizada em todos os seus componentes, persistindo algumas falhas pontuais na fluidez da comunicação ou na estrutura de certas rotinas. Ainda assim, a arquitetura modular baseada em herança e polimorfismo permitiu abstrair o hardware e cumprir as metas estabelecidas. Em suma, o trabalho resultou numa aplicação funcional que serviu como uma importante base de aprendizagem prática sobre os fundamentos de sistemas operativos.

Como perspetiva de evolução para o sistema desenvolvido, identifica-se a oportunidade de otimizar os mecanismos de sincronização para permitir uma reatividade ainda maior do robot, possivelmente explorando bloqueios de leitura/escrita que permitam monitorizar sensores em paralelo com movimentos não críticos. No âmbito da interface de utilizador, seria interessante evoluir a GUI_Gravador para incluir um editor visual de comandos, permitindo ao utilizador modificar ou reordenar ações gravadas num ficheiro antes da sua reprodução física.

No âmbito da interface de utilizador, seria interessante evoluir a classe GUI_Gravador para incluir um editor visual de comandos. Esta funcionalidade permitiria ao utilizador modificar, remover ou reordenar ações gravadas num ficheiro diretamente na interface antes da sua reprodução física. Adicionalmente, o sistema poderia ser expandido para suportar novos sensores do robot.

Bibliografia

[1] Jorge Pais. *Fundamentos de Sistemas Operativos*. Slides de curso. Versão 2. Ano letivo 2025-2026. 2025.

[2] Stack Overflow. *Reading a plain text file in Java*. https://stackoverflow.com/questions/4716503/reading-a-plain-text-file-in-java. Acedido em: 19-11-2025. 2011.

[3] GeeksforGeeks. *Java Program to Split a String by Space*. https://www.geeksforgeeks.org/java/java-split-string-by-space/. Acedido em: 19-11-2025.

[4] GeeksforGeeks. *BufferedReader Class in Java*. https://www.geeksforgeeks.org/java/java-io-bufferedreader-class-java/. Acedido em: 19-11-2025.

[5] Jorge Pais. *JAVA- Swing não é "Thread-Safe"*. Slides Auxiliares. 2025.

Código

Classe App

```java
import javax.swing.*;

public class App implements Runnable {
    private GUI gui;
    private GUI_Gravador gui_gravador;
    private Dados dados;

    public static void main(String[] args) {
        SwingUtilities.invokeLater(new App());
    }

    public void run() {
        gui_gravador = new GUI_Gravador(this);
        gui = new GUI(this);

        dados = new Dados(gui, gui_gravador);
        gui_gravador.setDados(dados);
        gui.setDados(dados);
        gui.setVisible(true);
    }

    public GUI_Gravador getGui_gravador() {
        return gui_gravador;
    }
}
```

Listing 1: *Código Classe App*

Classe BufferCircularComando

```java
import java.util.concurrent.Semaphore;

public class BufferCircularComando {
    private final int dimensaoBuffer = 16;
    private final Semaphore livres, ocupados, acesso;
    private Comando buffer[];
    private int putBuffer, getBuffer;

    public BufferCircularComando() {
        livres = new Semaphore(dimensaoBuffer);
        ocupados = new Semaphore(0);
        acesso = new Semaphore(1);

        putBuffer = getBuffer = 0;

        buffer = new Comando[dimensaoBuffer];
    }
```

```
18
19      public void inserirElemento(Comando c) {
20          try {
21              livres.acquire(); // bloqueia se buffer cheio
22              acesso.acquire(); // obtém exclusão mútua para modificar buffer
23              buffer[putBuffer] = c; // insere o comando na posição atual
24              putBuffer = ++putBuffer % dimensaoBuffer; // avança índice
                    circularmente
25              acesso.release(); // liberta mutex
26              ocupados.release(); // sinaliza que há mais um itém disponivel
27          } catch (InterruptedException e) {
28              Thread.currentThread().interrupt();
29          }
30      }
31
32      public Comando removerElemento() {
33          try {
34              ocupados.acquire(); // bloqueia se buffer vazio
35              acesso.acquire(); // exclusão mutua
36              Comando c = new Comando(buffer[getBuffer]); // copia
37              buffer[getBuffer] = null; // limpar posição
38              getBuffer = ++getBuffer % dimensaoBuffer;
39              acesso.release();
40              livres.release(); // liberta uma posição livre
41              return c;
42          } catch (InterruptedException e) {
43              // Interrupção durante espera -> devolver null para o consumidor
                    lidar
44              Thread.currentThread().interrupt();
45              return null;
46          }
47      }
48
49      // Limpar rapidamente o buffer (para PARAR imediato)
50      public void limparBuffer() {
51          try {
52              acesso.acquire();
53
54              for (int i = 0; i < dimensaoBuffer; i++) {
55                  buffer[i] = null;
56              }
57
58              int ocup = ocupados.drainPermits();
59              if (ocup > 0) {
60                  livres.release(ocup);
61              }
62              putBuffer = 0;
63              getBuffer = 0;
64          } catch (InterruptedException e) {
65              Thread.currentThread().interrupt();
66          } finally {
```

```
            acesso.release();
        }
    }
}
```

Listing 2: *Código Classe BufferCircularComando*

Classe Comando

```
public class Comando {
    public final static int ID_RETA = 0;
    public final static int ID_DIREITA = 1;
    public final static int ID_ESQUERDA = 2;
    public final static int ID_PARAR = 3;
    private int identificador; // identificador dos comandos
    private int arg1, arg2;
    private String descricao;

    public Comando(int identificador, int arg1, int arg2) {
        this.identificador = identificador;
        this.arg1 = arg1;
        this.arg2 = arg2;
        setDescricaoById();
    }

    public Comando(int identificador, int arg1) {
        this.identificador = identificador;
        this.arg1 = arg1;
        setDescricaoById();
    }

    public Comando(Comando c) { // construtor por cópia
        this.identificador = c.identificador;
        this.arg1 = c.arg1;
        this.arg2 = c.arg2;
        this.descricao = c.descricao;
    }

    public int getIdentificador() {
        return identificador;
    }

    public void setIdentificador(int identificador) {
        this.identificador = identificador;
    }

    public int getArg1() {
        return arg1;
    }

```

```
42      public void setArg1(int arg1) {
43          this.arg1 = arg1;
44      }
45
46      public int getArg2() {
47          return arg2;
48      }
49
50      public void setArg2(int arg2) {
51          this.arg2 = arg2;
52      }
53
54      @Override
55      public String toString() {
56          return "Comando: " + descricao;
57      }
58
59      private void setDescricaoById() {
60          switch (this.identificador) {
61              case ID_RETA:
62                  this.descricao = "RETA(" + arg1 + ")";
63                  break;
64              case ID_DIREITA:
65                  this.descricao = "DIREITA(raio=" + arg1 + ", angulo=" + arg2
                        + ")";
66                  break;
67              case ID_ESQUERDA:
68                  this.descricao = "ESQUERDA(raio=" + arg1 + ", angulo=" + arg2
                         + ")";
69                  break;
70              case ID_PARAR:
71                  this.descricao = "PARAR(" + (arg1 == 1 ? "true" : "false") +
                        ")";
72                  break;
73              default:
74                  this.descricao = "DESCONHECIDO";
75                  break;
76
77          }
78      }
79
80      public long executarComandos(IRobotLegoEV3 robot) {
81          switch (identificador) {
82              case Comando.ID_RETA: {
83                  int d = arg1;
84                  robot.Reta(d);
85                  return TempoMovimento.tempoReta(d);
86              }
87              case Comando.ID_DIREITA: {
88                  int raioD = arg1;
89                  int anguloD = arg2;
```

```
                robot.CurvarDireita(raioD, anguloD);
                return TempoMovimento.tempoCurvar(raioD, anguloD);
            }
            case Comando.ID_ESQUERDA: {
                int raioE = arg1;
                int anguloE = arg2;
                robot.CurvarEsquerda(raioE, anguloE);
                return TempoMovimento.tempoCurvar(raioE, anguloE);
            }
            case Comando.ID_PARAR: {
                boolean imediato = (arg1 != 0);
                robot.Parar(imediato);
                return TempoMovimento.getTempoComunicacao();
            }
            default:
                return 0;
        }
    }
}
```

Listing 3: *Código Classe Comando*

Classe CRobotLegoEV3

```
public class CRobotLegoEV3 implements IRobotLegoEV3{

    private RobotLegoEV3 robot;

    public CRobotLegoEV3() {
        this.robot = new RobotLegoEV3();
    }

    @Override
    public boolean OpenEV3(String s) {
        return robot.OpenEV3(s);
    }

    @Override
    public void CloseEV3() {
        robot.CloseEV3();
    }

    @Override
    public void Reta(int d) {
        robot.Reta(d);
    }

    @Override
    public void CurvarEsquerda(int raio, int angulo) {
        robot.CurvarEsquerda(raio, angulo);
```

```
    }

    @Override
    public void CurvarDireita(int raio, int angulo) {
        robot.CurvarDireita(raio, angulo);
    }

    @Override
    public void Parar(boolean p) {
        robot.Parar(p);
    }

    @Override
    public int SensorToque(int toque) {
        return  robot.SensorToque(toque);
    }

}
```

Listing 4: *Código Classe CRobotLegoEV3*

Classe Dados

```
import java.util.ArrayList;

public class Dados {
    private String nomeRobot;
    private GUI gui;
    private GUI_Gravador guiGravador;
    private ArrayList<String> texto;
    private int raio, angulo, distancia;
    private boolean onOff;
    private MyRobotSync robot;

    private EvitarObstaculos evitarObstaculos;
    private MovAleatorios movAleatorios;
    private Gravador gravador;

    public Dados(GUI gui, GUI_Gravador guiGravador) {
        nomeRobot = "EV8";
        this.gui = gui;
        this.guiGravador = guiGravador;
        texto = new ArrayList<String>();
        raio = 20;
        angulo = 90;
        distancia = 33;
        onOff = false;
        robot = new MyRobotSync();

        // tarefa mov aleatorios
```

```
28          movAleatorios = new MovAleatorios(this);
29          movAleatorios.start();
30
31          // tarefa evitar obstaculos
32          evitarObstaculos = new EvitarObstaculos(this);
33          evitarObstaculos.start();
34
35          // tarefa gravador
36          gravador = new Gravador(this);
37          gravador.start();
38      }
39
40      public MovAleatorios getMovAleatorios() {
41          return movAleatorios;
42      }
43
44      public EvitarObstaculos getEvitarObstaculos() {
45          return evitarObstaculos;
46      }
47
48      public String getNomeRobot() {
49          return nomeRobot;
50      }
51
52      public void setNomeRobot(String nomeRobot) {
53          this.nomeRobot = nomeRobot;
54      }
55
56      public GUI getGui() {
57          return gui;
58      }
59
60      public void setGui(GUI gui) {
61          this.gui = gui;
62      }
63
64      public ArrayList<String> getTexto() {
65          return texto;
66      }
67
68      public void setTexto(ArrayList<String> texto) {
69          this.texto = texto;
70      }
71
72      public int getRaio() {
73          return raio;
74      }
75
76      public void setRaio(int raio) {
77          this.raio = raio;
78      }
```

```
79
80      public int getAngulo() {
81          return angulo;
82      }
83
84      public void setAngulo(int angulo) {
85          this.angulo = angulo;
86      }
87
88      public int getDistancia() {
89          return distancia;
90      }
91
92      public void setDistancia(int distancia) {
93          this.distancia = distancia;
94      }
95
96      public boolean isOnOff() {
97          return onOff;
98      }
99
100     public void setOnOff(boolean onOff) {
101         this.onOff = onOff;
102     }
103
104     public MyRobotSync getRobot() {
105         return robot;
106     }
107
108     public void setRobot(MyRobotSync robot) {
109         this.robot = robot;
110     }
111
112     public Gravador getGravador() {
113         return gravador;
114     }
115
116     public GUI_Gravador getGuiGravador() {
117         return guiGravador;
118     }
119
120     public void setGravador(Gravador gravador) {
121         this.gravador = gravador;
122     }
123
124     /**
125      * Método para encaminhar os comandos para o gravdor quando acionarmos o
            modo*/
126     public void registarComandos(Comando c) {
127         if (gravador != null) {
128             gravador.gravar(c);
```

```
        }
    }
}
```

Listing 5: *Código Classe Dados*

Classe EvitarObstaculos

```
import java.util.Random;

/**
 * A tarefa "Evitar Obstá"culo deve de estar a testar o sensor de toque de 50 em 50 ms, e
 * quando este ficar ativo deve imediatamente parar o robot, depois fazer recuar o robot de 20
 * cm, e depois fazer uma curva à esquerda ou à direita (com escolha aleatória) de 90 graus.
 * Durante estas ações nem o utilizador nem as outras tarefas devem poder movimentar o robot.
 */
public class EvitarObstaculos extends Tarefa {
    private final int ESPERA_SENSOR = 50;
    private Dados dados;
    private MyRobotSync robot;
    private Random random;

    public EvitarObstaculos(Dados dados) {
        this.dados = dados;
        random = new Random();
        robot = dados.getRobot();
    }

    @Override
    void trabalharAtividade() throws InterruptedException {
        int toque;
        try {
            dados.getRobot().entrar();
            toque = robot.SensorToque(RobotLegoEV3.S_1);
        } finally {
            dados.getRobot().sair();
        }
        switch (toque) {
            case 0:
                Thread.sleep(ESPERA_SENSOR);
                break;
            case 1:

                dados.getGui().print("Evitar Obstáculos: Obstáculo Detectado");

```

```
38                  // se os mov aleatorios estiverem ATIVOS e a FUNCIONAR
39                  if (dados.getMovAleatorios() != null && dados.
                        getMovAleatorios().estado == Estados.TRABALHAR) {
40                      dados.getGui().print("Evitar Obstáculos: Interromper
                            Movimentos Aleatórios");
41                      dados.getMovAleatorios().interrupt(); // interromper (
                            mudar)
42                  }
43                  try {
44                      dados.getRobot().entrar();
45                      dados.getGui().print("Evitar Obstáculos: Semáforo
                            adquirido");
46
47                      dados.getRobot().Parar(true); // parar imediatamente
48                      dados.getGui().print("Evitar Obstaculos: Parar.");
49
50                      Thread.sleep(TempoMovimento.getTempoComunicacao());
51
52                      // reta
53                      int distancia = -20;
54                      dados.getRobot().Reta(distancia);
55                      dados.registarComandos(new Comando(Comando.ID_RETA,
                            distancia));
56                      dados.getGui().print("Evitar Obstáculos: Reta com distâ
                            ncia: " + distancia);
57                      Thread.sleep(TempoMovimento.tempoReta(distancia));
58
59                      int escolha = random();
60
61                      switch (escolha) {
62                          // esquerda
63                          case 0:
64                              dados.getRobot().CurvarEsquerda(0, 90);
65                              dados.registarComandos(new Comando(Comando.
                                    ID_ESQUERDA, 0, 90));
66                              dados.getGui().print("Evitar Obstáculos: Curva
                                    Esquerda com raio: 0, e angulo: 90º. ");
67                              Thread.sleep(TempoMovimento.tempoCurvar(0, 90));
68
69                              break;
70                          case 1:
71                              dados.getRobot().CurvarDireita(0, 90);
72                              dados.registarComandos(new Comando(Comando.
                                    ID_DIREITA, 0, 90));
73                              dados.getGui().print("Evitar Obstáculos: Curva
                                    Direita com raio: 0, e angulo: 90º. ");
74                              Thread.sleep(TempoMovimento.tempoCurvar(0, 90));
75
76                              break;
77                      }
78                      dados.getRobot().Parar(false);
```

```
                    break;

                } finally {
                    dados.getRobot().sair();
                    dados.getGui().print("Evitar Obstáculos: Semaforo
                        Liberado");
                }
            }

            Thread.sleep(ESPERA_SENSOR);
        }

        private int random() {
            return random.nextInt(2);
        }

}
```

Listing 6: *Código Classe EvitarObstaculos*

Classe Gravador

```
import java.io.*;

/**
 * Esta tarefa tem a possibilidade de
 * trabalhar com o mesmo robot ou com um robot diferente das outras tarefas.
 * A manipulação do ficheiro deve obrigatoriamente ser efetuada com as
     classes de
 * InputStream, Reader, OutputStream e Writer do java.io.
 */
public class Gravador extends Tarefa {
    private Dados dados;

    // gravação
    private volatile boolean gravar;
    private String caminhoGravar;
    private OutputStream os;
    private Writer writer;

    // reprodução
    private volatile boolean reproduzir;
    private String caminhoReproduzir;

    public Gravador(Dados dados) {
        this.dados = dados;
        gravar = false;
        reproduzir = false;
    }

```

```
28      public synchronized boolean isGravar() {
29          return gravar;
30      }
31
32      public synchronized void iniciarGravar(String caminhoFicheiro) {
33          // se estiver a gravar
34          if (gravar) {
35              dados.getGuiGravador().print("Gravador: Gravação ativa");
36              return;
37          }
38          // tratamento de erros
39          if (caminhoFicheiro == null || caminhoFicheiro.trim().isEmpty()) {
40              dados.getGuiGravador().print("Gravador: Caminho do ficheiro invá
                    lido");
41              return;
42          }
43
44          try {
45              caminhoGravar = caminhoFicheiro;
46
47              os = new FileOutputStream(caminhoGravar, false);
48              writer = new OutputStreamWriter(os, "UTF-8");
49              gravar = true;
50              dados.getGuiGravador().print("Gravador: gravação iniciada em " +
                    caminhoGravar);
51          } catch (IOException e) {
52              dados.getGuiGravador().print("Gravador: erro a iniciar gravação:
                    " + e.getMessage());
53              fecharWritter();
54          }
55      }
56
57      public synchronized void pararGravacao() {
58          if (!gravar) {
59              dados.getGuiGravador().print("Gravador: não está a gravar");
60              return;
61          }
62          fecharWritter();
63          gravar = false;
64          dados.getGuiGravador().print("Gravador: gravação acabada");
65
66
67      }
68
69      public synchronized void pararReproducao() {
70          reproduzir = false;
71          estado = Estados.DORMIR;
72          this.interrupt();
73          try {
74              dados.getRobot().Parar(false);
75          } catch (Exception ignore) {
```

```
76          }
77          dados.getGuiGravador().print("Gravador: reproduzir parado");
78          dados.getGuiGravador().resetBotaoReprod();
79      }
80
81      public synchronized void gravar(Comando comando) {
82          if (!gravar || writer == null || comando == null) return;
83          try {
84              String linha = encode(comando);
85              writer.write(linha);
86              writer.write("\n");
87              writer.flush();
88          } catch (IOException e) {
89              dados.getGuiGravador().print("Gravador: erro a escrever comando:
                    " + e.getMessage());
90          }
91      }
92
93      public synchronized void reproduzir(String caminhoFicheiro) {
94          if (reproduzir) {
95              dados.getGuiGravador().print("Gravador: já está a reproduzir");
96              return;
97          }
98
99          if (caminhoFicheiro == null || caminhoFicheiro.trim().isEmpty()) {
100             dados.getGuiGravador().print("Gravador: caminho do ficheiro invá
                    lido");
101             return;
102         }
103
104         // ativar tarefa para trabalhar
105         this.caminhoReproduzir = caminhoFicheiro;
106         this.reproduzir = true;
107         ativar();
108     }
109
110     public boolean isReproduzir() {
111         return reproduzir;
112     }
113
114     /**
115      * Método auxiliar para de comando para texto
116      */
117     private String encode(Comando comando) {
118         switch (comando.getIdentificador()) {
119             case Comando.ID_RETA:
120                 return "RETA " + comando.getArg1();
121             case Comando.ID_DIREITA:
122                 return "DIREITA " + comando.getArg1() + " " + comando.getArg2
                        ();
123             case Comando.ID_ESQUERDA:
```

```
124                 return "ESQUERDA " + comando.getArg1() + " " + comando.
                        getArg2();
125             case Comando.ID_PARAR:
126                 return "PARAR " + (comando.getArg1() != 0 ? 1 : 0);
127             default:
128                 return "desconhecido " + comando.toString();
129         }
130     }
131
132     private Comando decode(String linha) {
133         try {
134             String[] comando = linha.split("\\s+"); // vamos separar um array
                     de strings
135             // Ex: RETA 10 -> ["RETA", "10"]
136             String op = comando[0].toUpperCase();
137             switch (op) {
138                 case "RETA":
139                     return new Comando(Comando.ID_RETA, Integer.parseInt(
                            comando[1]));
140                 case "DIREITA":
141                     return new Comando(Comando.ID_DIREITA, Integer.parseInt(
                            comando[1]), Integer.parseInt(comando[2]));
142                 case "ESQUERDA":
143                     return new Comando(Comando.ID_ESQUERDA, Integer.parseInt(
                            comando[1]), Integer.parseInt(comando[2]));
144                 case "PARAR":
145                     return new Comando(Comando.ID_PARAR, Integer.parseInt(
                            comando[1]));
146                 default:
147                     return null;
148             }
149         } catch (Exception e) {
150             return null;
151         }
152     }
153
154     @Override
155     void trabalharAtividade() {
156         // executar SÓ se estiver em modo de reprodução
157         if (!reproduzir) {
158             estado = Estados.DORMIR;
159             return;
160         }
161
162         // tratamento de erros
163         if (!dados.isOnOff()) {
164             dados.getGuiGravador().print("Gravador: ligue o robot primeiro");
165             reproduzir = false;
166             estado = Estados.DORMIR;
167             return;
168         }
```

```
169
170         // REPRODUÇÃO
171         dados.getGuiGravador().print("Gravador: iniciar reprodução de " +
                caminhoReproduzir);
172
173         try (InputStream is = new FileInputStream(caminhoReproduzir);
174              Reader ir = new InputStreamReader(is, "UTF-8");
175              BufferedReader br = new BufferedReader(ir)) {
176
177             dados.getGuiGravador().print("Gravador: Semáforo adquirido");
178
179             String linha;
180             while ((linha = br.readLine()) != null && estado == Estados.
                    TRABALHAR && reproduzir) {
181
182                 if (Thread.interrupted()) break;
183
184                 linha = linha.trim();
185                 if (linha.isEmpty()) continue;
186
187                 Comando comando = decode(linha);
188                 if (comando == null) {
189                     dados.getGuiGravador().print("Gravador: linha inválida: "
                            + linha);
190                     continue;
191                 }
192                 executarComando(comando);
193             }
194
195             // no fim parar de forma não imediata para acabar a sequência
196             dados.getGuiGravador().print("Gravador: Parar(false)");
197             dados.getRobot().Parar(false);
198             dados.getGuiGravador().print("Gravador: reprodução concluída");
199         } catch (IOException e) {
200             dados.getGuiGravador().print("Gravador: erro ao ler ficheiro: " +
                     e.getMessage());
201         } finally {
202             reproduzir = false;
203             estado = Estados.DORMIR;
204             dados.getGuiGravador().resetBotaoReprod();
205         }
206     }
207
208     private void executarComando(Comando comando) {
209         long espera = comando.executarComandos(dados.getGuiGravador().
                getRobotAtual());
210
211         dados.getGuiGravador().print("Gravador: " + comando.toString());
212         if (espera > 0) {
213             dados.getGuiGravador().print("Gravador: espera " + espera + " ms"
                    );
```

```
            try {
                Thread.sleep(espera);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
    private synchronized void fecharWritter() {
        try {
            if (writer != null) writer.close();
        } catch (IOException e) {

        } finally {
            writer = null;
        }

        try {
            if (os != null) os.close();
        } catch (IOException e) {
        } finally {
            os = null;
        }
    }
}
```

Listing 7: *Código Classe Gravador*

Classe GUI

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.text.DefaultCaret;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

public class GUI extends JFrame {

    private static final long serialVersionUID = 1L;
    private Dados dados;

    private JPanel contentPane;
    private JTextField raioField;
    private JTextField anguloField;
    private JTextField distanciaField;
    private JTextField robotField;
    private JTextArea consolaTextArea;

```

```
22      private JButton frenteBtn;
23      private JButton trasBtn;
24      private JButton pararBtn;
25      private JButton esquerdaBtn;
26      private JButton direitaBtn;
27      private JSpinner numSpinner;
28      private JRadioButton movAleatoriosRadioBtn;
29      private JCheckBox tipoRobotChkBox;
30      private JButton limparBtn;
31      private JCheckBox imprimirChkBox;
32      private boolean robotGuiLigado;
33
34      public boolean isRobotGuiLigado() {
35          return robotGuiLigado;
36      }
37
38      public void setRobotGuiLigado(boolean robotGuiLigado) {
39          this.robotGuiLigado = robotGuiLigado;
40      }
41
42      /**
43       * Create the frame.
44       */
45      public GUI(App app) {
46
47          GUI_Gravador gui_gravador = app.getGui_gravador();
48          // Iniciar Janela
49          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
50          setBounds(100, 100, 470, 610);
51          setTitle("GUI Principal");
52          contentPane = new JPanel();
53          contentPane.setBackground(new Color(254, 255, 255));
54          contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
55          setContentPane(contentPane);
56          contentPane.setLayout(null);
57
58          JLabel raioLabel = new JLabel("Raio");
59          raioLabel.setBounds(6, 6, 27, 16);
60          contentPane.add(raioLabel);
61
62          raioField = new JTextField();
63          raioField.addActionListener(new ActionListener() {
64              public void actionPerformed(ActionEvent arg0) {
65                  // Meter raio
66                  dados.setRaio(Integer.parseInt(raioField.getText()));
67              }
68          });
69          raioField.setBounds(45, 1, 33, 26);
70          contentPane.add(raioField);
71          raioField.setColumns(10);
72
```

```
73          JLabel anguloLabel = new JLabel("Ângulo");
74          anguloLabel.setBounds(90, 6, 45, 16);
75          contentPane.add(anguloLabel);
76
77          anguloField = new JTextField();
78          anguloField.addActionListener(new ActionListener() {
79              public void actionPerformed(ActionEvent arg0) {
80                  // Meter ângulo
81                  dados.setAngulo(Integer.parseInt(anguloField.getText()));
82              }
83          });
84          anguloField.setBounds(147, 1, 33, 26);
85          contentPane.add(anguloField);
86          anguloField.setColumns(10);
87
88          JLabel distanciaLabel = new JLabel("Distância");
89          distanciaLabel.setBounds(190, 6, 61, 16);
90          contentPane.add(distanciaLabel);
91
92          distanciaField = new JTextField();
93          distanciaField.addActionListener(new ActionListener() {
94              public void actionPerformed(ActionEvent arg0) {
95                  // Meter distância
96                  dados.setDistancia(Integer.parseInt(distanciaField.getText())
                        );
97              }
98          });
99          distanciaField.setColumns(10);
100         distanciaField.setBounds(263, 1, 33, 26);
101         contentPane.add(distanciaField);
102
103         JLabel robotLabel = new JLabel("Robot");
104         robotLabel.setBounds(339, 6, 45, 16);
105         contentPane.add(robotLabel);
106
107         robotField = new JTextField();
108         robotField.addActionListener(new ActionListener() {
109             public void actionPerformed(ActionEvent arg0) {
110                 // Meter nome do Robot
111                 dados.setNomeRobot(robotField.getText());
112             }
113         });
114         robotField.setColumns(10);
115         robotField.setBounds(387, 1, 33, 26);
116         contentPane.add(robotField);
117
118         JCheckBox ligarChkBox = new JCheckBox("Ligar");
119         ligarChkBox.addActionListener(new ActionListener() {
120             public void actionPerformed(ActionEvent arg0) {
121                 // Ligar/Desligar Robot
122                 if (ligarChkBox.isSelected()) { // se selecionarmos o botão
```

```
123                     dados.getRobot().OpenEV3(dados.getNomeRobot());  // se
                            conseguirmos obter o nome do robot e ligar
124                     setRobotGuiLigado(true);
125                     dados.setOnOff(true);
126                     dados.getRobot().start();
127                     print("Sucesso na abertura das Streams");
128                 } else {
129                     // se desligar, parar a tarefa de mov aleatorios se
                            estiver ativa
130                     if (movAleatoriosRadioBtn.isSelected()) {
131                         movAleatoriosRadioBtn.setSelected(false);
132                         if (dados.getMovAleatorios() != null) {
133                             dados.getMovAleatorios().desativar();
134                         }
135                     }
136                     dados.getRobot().CloseEV3();
137                     dados.setOnOff(false);
138                     setRobotGuiLigado(false);
139                     //dados.getRobot().stop();
140                     ligarChkBox.setSelected(false);
141                     print("Conexão encerrada");
142                 }
143             }
144         });
145         ligarChkBox.setBounds(322, 34, 73, 23);
146         contentPane.add(ligarChkBox);
147 
148         frenteBtn = new JButton("FRENTE");
149         frenteBtn.setBounds(156, 55, 117, 29);
150         frenteBtn.addActionListener(new ActionListener() {
151             public void actionPerformed(ActionEvent e) {
152                 Runnable frenteTarefa = new Runnable() {
153                     @Override
154                     public void run() {
155                         frenteAction();
156                     }
157                 };
158                 new Thread(frenteTarefa).start();
159             }
160         });
161         contentPane.add(frenteBtn);
162 
163         pararBtn = new JButton("PARAR");
164         pararBtn.setBounds(156, 84, 117, 29);
165         pararBtn.addActionListener(new ActionListener() {
166             public void actionPerformed(ActionEvent e) {
167                 dados.getRobot().Parar(true);
168                 dados.registarComandos(new Comando(Comando.ID_PARAR, 1));
169                 print("Comando enviado: Parar (imediato)");
170             }
171         });
```

```
172         contentPane.add(pararBtn);
173 
174         trasBtn = new JButton("TRÁS");
175         trasBtn.setBounds(156, 114, 117, 29);
176         trasBtn.addActionListener(new ActionListener() {
177             public void actionPerformed(ActionEvent e) {
178                 Runnable trasTarefa = new Runnable() {
179                     @Override
180                     public void run() {
181                         trasAction();
182                     }
183                 };
184                 new Thread(trasTarefa).start();
185             }
186         });
187         contentPane.add(trasBtn);
188 
189         direitaBtn = new JButton("DIREITA");
190         direitaBtn.setBounds(267, 84, 117, 29);
191         direitaBtn.addActionListener(new ActionListener() {
192             public void actionPerformed(ActionEvent e) {
193                 Runnable direitaTarefa = new Runnable() {
194                     @Override
195                     public void run() {
196                         direitaAction();
197                     }
198                 };
199                 new Thread(direitaTarefa).start();
200             }
201         });
202         contentPane.add(direitaBtn);
203 
204         esquerdaBtn = new JButton("ESQUERDA");
205         esquerdaBtn.setBounds(45, 84, 117, 29);
206         esquerdaBtn.addActionListener(new ActionListener() {
207             public void actionPerformed(ActionEvent e) {
208                 Runnable esquerdaTarefa = new Runnable() {
209                     @Override
210                     public void run() {
211                         esquerdaAction();
212                     }
213                 };
214                 new Thread(esquerdaTarefa).start();
215             }
216         });
217         contentPane.add(esquerdaBtn);
218 
219         tipoRobotChkBox = new JCheckBox("Simulado/Real");
220         tipoRobotChkBox.setBounds(10, 155, 115, 23);
221         tipoRobotChkBox.addActionListener(new ActionListener() {
222             public void actionPerformed(ActionEvent e) {
```

```
223                 if (tipoRobotChkBox.isSelected()) {
224                     print("Modo Selecionado: Robot Simulado");
225                     dados.getRobot().setRobot(new RobotSimulado());
226                 } else {
227                     print("Modo Selecionado: Robot Real");
228                     dados.getRobot().setRobot(new CRobotLegoEV3());
229                 }
230             }
231         });
232         contentPane.add(tipoRobotChkBox);
233 
234         numSpinner = new JSpinner();
235         numSpinner.setBounds(191, 155, 34, 26);
236         contentPane.add(numSpinner);
237 
238         JLabel numeroLabel = new JLabel("Número");
239         numeroLabel.setBounds(135, 160, 61, 16);
240         contentPane.add(numeroLabel);
241 
242         movAleatoriosRadioBtn = new JRadioButton("Movimentos Aleatórios");
243         movAleatoriosRadioBtn.setBounds(277, 134, 182, 16);
244         movAleatoriosRadioBtn.addActionListener(new ActionListener() {
245             public void actionPerformed(ActionEvent e) {
246                 if (movAleatoriosRadioBtn.isSelected()) {
247                     if (!dados.isOnOff()) {
248                         print("Ligar robot primeiro!");
249                         movAleatoriosRadioBtn.setSelected(false);
250                     }
251                     print("Movimentos Aleatórios Selecionado");
252                     dados.getMovAleatorios().ativar();
253                 } else {
254                     print("Movimentos Aleatórios Desativado");
255                     dados.getMovAleatorios().desativar();
256                     dados.getRobot().Parar(false);
257                 }
258             }
259         });
260         contentPane.add(movAleatoriosRadioBtn);
261 
262         JLabel consolaLabel = new JLabel("Consola");
263         consolaLabel.setBounds(6, 280, 61, 16);
264         contentPane.add(consolaLabel);
265 
266 
267 
268         JRadioButton evitarRadioBtn = new JRadioButton("Evitar Obstáculos");
269         evitarRadioBtn.setBounds(277, 162, 182, 16);
270         evitarRadioBtn.addActionListener(new ActionListener() {
271             public void actionPerformed(ActionEvent e) {
272                 if (evitarRadioBtn.isSelected()) {
273                     if (!dados.isOnOff()) {
```

```
                    print("Ligar robot primeiro!");
                    evitarRadioBtn.setSelected(false);
                }
                print("Evitar Objetos Selecionado");
                dados.getEvitarObstaculos().ativar();
            } else {
                print("Evitar Objetos Desativado");
                dados.getEvitarObstaculos().desativar();
            }
        }
    });
    contentPane.add(evitarRadioBtn);

    JRadioButton gravadorRadioBtn = new JRadioButton("Gravador");
    gravadorRadioBtn.setBounds(277, 190, 182, 16);
    gravadorRadioBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            if (gravadorRadioBtn.isSelected()) {
                if (!dados.isOnOff()) {
                    print("Ligar robot primeiro!");
                    gravadorRadioBtn.setSelected(false);
                } else {

                print("Gravador Selecionado");
                gui_gravador.setVisible(true);
                }

            } else {
                print("Gravador Desativado");
                gui_gravador.setVisible(false);
            }
        }
    });
    contentPane.add(gravadorRadioBtn);

    JScrollPane consolaScrollPane = new JScrollPane();
    consolaScrollPane.setBounds(6, 300, 455, 230);
    contentPane.add(consolaScrollPane);
    consolaTextArea = new JTextArea();
    DefaultCaret caret = (DefaultCaret) consolaTextArea.getCaret();
    caret.setUpdatePolicy(DefaultCaret.ALWAYS_UPDATE);
    consolaScrollPane.setViewportView(consolaTextArea);



    limparBtn = new JButton("Limpar");
    limparBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            consolaTextArea.setText(null); // Limpa a consola
        }
    });
```

```
325         limparBtn.setBounds(120, 540, 117, 29);
326         contentPane.add(limparBtn);
327 
328         imprimirChkBox = new JCheckBox("Imprimir");
329         imprimirChkBox.setSelected(true);
330         imprimirChkBox.addActionListener(new ActionListener() {
331             public void actionPerformed(ActionEvent e) {
332                 if (imprimirChkBox.isSelected()) {
333                     print("Impressão na consola ATIVADA.");
334                 }
335             }
336         });
337         imprimirChkBox.setBounds(250, 542, 100, 23);
338         contentPane.add(imprimirChkBox);
339 
340         setVisible(true);
341     }
342 
343     public JSpinner getNumSpinner() {
344         return numSpinner;
345     }
346 
347     public void setDados(Dados dados) {
348         this.dados = dados;
349         // valores pre definidos gui
350         raioField.setText(Integer.toString(dados.getRaio()));
351         anguloField.setText(Integer.toString(dados.getAngulo()));
352         distanciaField.setText(Integer.toString(dados.getDistancia()));
353         robotField.setText(dados.getNomeRobot());
354     }
355 
356     public void print(String s) {
357         if (imprimirChkBox == null) return;
358         Runnable runnable = () -> {
359             if (imprimirChkBox.isSelected()) {
360                 consolaTextArea.append(s + "\n");
361             }
362         };
363         if (SwingUtilities.isEventDispatchThread()) {
364             runnable.run();
365         } else {
366             SwingUtilities.invokeLater(runnable);
367         }
368     }
369 
370     private void frenteAction() {
371         try {
372             // TODO confirmar se é preciso tirar estas entradas de semáforo j
                    á que fazemos no MyRobotSync
373             dados.getRobot().entrar();
374             dados.getRobot().Reta(dados.getDistancia());
```

```
375             dados.registarComandos(new Comando(Comando.ID_RETA, dados.
                    getDistancia()));
376             print("Comando enviado: Frente com distância " + dados.
                    getDistancia());
377             print("Espera: " + TempoMovimento.tempoReta(dados.getDistancia())
                    );
378             // TODO confirmar com stor
379             dados.registarComandos(new Comando(Comando.ID_PARAR, 0));
380             dados.getRobot().Parar(false);
381         } catch (InterruptedException ex) {
382             ex.printStackTrace();
383         } finally {
384             dados.getRobot().sair();
385         }
386     }
387
388     private void trasAction() {
389         try {
390             dados.getRobot().entrar();
391             dados.getRobot().Reta(dados.getDistancia() * -1);
392             dados.registarComandos(new Comando(Comando.ID_RETA, dados.
                    getDistancia() * -1));
393             print("Comando enviado: Trás com distância: " + dados.
                    getDistancia());
394             print("Espera: " + TempoMovimento.tempoReta(dados.getDistancia())
                    );
395             dados.registarComandos(new Comando(Comando.ID_PARAR, 0));
396             dados.getRobot().Parar(false);
397         } catch (InterruptedException ex) {
398             ex.printStackTrace();
399         } finally {
400             dados.getRobot().sair();
401         }
402     }
403
404     private void direitaAction() {
405         try {
406             dados.getRobot().entrar();
407             dados.getRobot().CurvarDireita(dados.getRaio(), dados.getAngulo()
                    );
408             dados.registarComandos(new Comando(Comando.ID_DIREITA, dados.
                    getRaio(), dados.getAngulo()));
409             print("Comando enviado: Curvar à Direita com ângulo " + dados.
                    getAngulo() + " e raio " + dados.getRaio());
410             print("Espera: " + TempoMovimento.tempoCurvar(dados.getRaio(),
                    dados.getAngulo()) + " ms");
411             dados.registarComandos(new Comando(Comando.ID_PARAR, 0));
412             dados.getRobot().Parar(false);
413         } catch (InterruptedException ex) {
414             ex.printStackTrace();
415         } finally {
```

```
            dados.getRobot().sair();
        }
    }

    private void esquerdaAction() {
        try {
            dados.getRobot().entrar();
            dados.getRobot().CurvarEsquerda(dados.getRaio(), dados.getAngulo
                ());
            dados.registarComandos(new Comando(Comando.ID_ESQUERDA, dados.
                getRaio(), dados.getAngulo()));
            print("Comando enviado: Curvar à Esquerda com ângulo " + dados.
                getAngulo() + " e raio " + dados.getRaio());
            print("Espera: " + TempoMovimento.tempoCurvar(dados.getRaio(),
                dados.getAngulo()) + " ms");
            dados.registarComandos(new Comando(Comando.ID_PARAR, 0));
            dados.getRobot().Parar(false);
        } catch (InterruptedException ex) {
            ex.printStackTrace();
        } finally {
            dados.getRobot().sair();
        }


    }
}
```

Listing 8: *Código Classe GUI*

Classe GUIGravador

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.text.DefaultCaret;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

public class GUI_Gravador extends JFrame {
    private static final long serialVersionUID = 1L;
    private Dados dados;

    private JPanel contentPane;
    private JTextField raioField;
    private JTextField anguloField;
    private JTextField distanciaField;
    private JTextField robotField;
    private JTextArea consolaTextArea;
```

```
20      private JButton frenteBtn;
21      private JButton trasBtn;
22      private JButton pararBtn;
23      private JButton esquerdaBtn;
24      private JButton direitaBtn;
25      private JCheckBox tipoRobotChkBox;
26      private JPanel gravadorPanel;
27      private JTextField ficheiroField;
28      private JButton browseBtn;
29      private JButton gravarBtn;
30      private JButton reproduzirBtn;
31      private JButton limparBtn;
32      private JCheckBox imprimirChkBox;
33      private JCheckBox ligarChkBox;
34
35      private MyRobotSync robotLocal;
36      private boolean ligado = false;
37      private String nomeAtual;
38
39      /**
40       * Create the frame.
41       */
42      public GUI_Gravador(App app) {
43
44          // Iniciar Janela
45          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
46          setBounds(100, 100, 470, 610);
47          setTitle("GUI Gravador");
48          contentPane = new JPanel();
49          contentPane.setBackground(new Color(254, 255, 255));
50          contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
51          setContentPane(contentPane);
52          contentPane.setLayout(null);
53
54          JLabel raioLabel = new JLabel("Raio");
55          raioLabel.setBounds(6, 6, 27, 16);
56          contentPane.add(raioLabel);
57
58          raioField = new JTextField();
59          raioField.addActionListener(new ActionListener() {
60              public void actionPerformed(ActionEvent arg0) {
61                  // Meter raio
62                  dados.setRaio(Integer.parseInt(raioField.getText()));
63              }
64          });
65          raioField.setBounds(45, 1, 33, 26);
66          contentPane.add(raioField);
67          raioField.setColumns(10);
68
69          JLabel anguloLabel = new JLabel("Ângulo");
70          anguloLabel.setBounds(90, 6, 45, 16);
```

```
        contentPane.add(anguloLabel);

        anguloField = new JTextField();
        anguloField.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent arg0) {
                // Meter ângulo
                dados.setAngulo(Integer.parseInt(anguloField.getText()));
            }
        });
        anguloField.setBounds(147, 1, 33, 26);
        contentPane.add(anguloField);
        anguloField.setColumns(10);

        JLabel distanciaLabel = new JLabel("Distância");
        distanciaLabel.setBounds(190, 6, 61, 16);
        contentPane.add(distanciaLabel);

        distanciaField = new JTextField();
        distanciaField.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent arg0) {
                // Meter distância
                dados.setDistancia(Integer.parseInt(distanciaField.getText())
                    );
            }
        });
        distanciaField.setColumns(10);
        distanciaField.setBounds(263, 1, 33, 26);
        contentPane.add(distanciaField);

        JLabel robotLabel = new JLabel("Robot");
        robotLabel.setBounds(339, 6, 45, 16);
        contentPane.add(robotLabel);

        robotField = new JTextField();
        robotField.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent arg0) {
                // Meter nome do Robot
                String novoNome = robotField.getText().trim();

                if (novoNome.equals(nomeAtual)) return;

                if (ligado) {
                    boolean eraPartilhado = isPartilhado();

                    // fechar robot local antigo se existir
                    if (!eraPartilhado && robotLocal != null) {
                        try {
                            robotLocal.Parar(true);
                            robotLocal.CloseEV3();
                            robotLocal.stop();
                        } catch (Exception ignore) {
```

```
                        robotLocal = null;
                    }
                    nomeAtual = novoNome;

                    // verificar novo robot
                    if (isPartilhado()) {
                        if (dados.isOnOff()) {
                            print("A usar robot partilhado: " + nomeAtual
                                );
                        } else {
                            print("Robot partilhado não está ligado");
                        }
                    } else {
                        // criar e ligar novo robot local
                        robotLocal = new MyRobotSync();
                        robotLocal.OpenEV3(nomeAtual);
                        robotLocal.start();
                        print("Robot mudado para_ " + nomeAtual);
                    }
                } else {
                    nomeAtual = novoNome;
                    print("Nome definido: " + nomeAtual + "(ligar para
                        ativar)");
                }
            }
        }
    });
    robotField.setColumns(10);
    robotField.setBounds(387, 1, 33, 26);
    contentPane.add(robotField);

    ligarChkBox = new JCheckBox("Ligar");
    ligarChkBox.setSelected(false);
    ligarChkBox.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent arg0) {
            if (ligarChkBox.isSelected()) {
                nomeAtual = robotField.getText().trim();
                if (isPartilhado()) {
                    if (dados.isOnOff()) {
                        ligado = true;
                        print("A usar robot partilhado" + nomeAtual);
                    } else {
                        print("Ligar o robot na GUI principal primeiro");
                        ligarChkBox.setSelected(false);
                    }
                } else {
                    // criar robot local se não existe
                    if (robotLocal == null) {
                        robotLocal = new MyRobotSync();
                    }
                    robotLocal.OpenEV3(nomeAtual);
```

```
                    robotLocal.start();
                    ligado = true;
                    print("Robot Ligado: " + nomeAtual);
                }
            } else {
                // desligar - se for local, fechar
                // se for partilhado só parar de enviar
                if (!isPartilhado() && robotLocal != null) {
                    try {
                        robotLocal.Parar(true);
                        robotLocal.CloseEV3();
                        robotLocal.stop();
                    } catch (Exception ignored) {
                        robotLocal = null;
                    }
                    ligado = false;
                    print("Comandos desativados");
                }
            }
        }
    });
    ligarChkBox.setBounds(322, 34, 73, 23);
    contentPane.add(ligarChkBox);

    frenteBtn = new JButton("FRENTE");
    frenteBtn.setBounds(156, 55, 117, 29);
    frenteBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            Runnable frenteTarefa = new Runnable() {
                @Override
                public void run() {
                    frenteAction();
                }
            };
            new Thread(frenteTarefa).start();
        }
    });
    contentPane.add(frenteBtn);

    pararBtn = new JButton("PARAR");
    pararBtn.setBounds(156, 84, 117, 29);
    pararBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            getRobotAtual().Parar(true);
            dados.registarComandos(new Comando(Comando.ID_PARAR, 1));
            print("Comando enviado: Parar (imediato)");
        }
    });
    contentPane.add(pararBtn);

    trasBtn = new JButton("TRÁS");
```

```
        trasBtn.setBounds(156, 114, 117, 29);
        trasBtn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                Runnable trasTarefa = new Runnable() {
                    @Override
                    public void run() {
                        trasAction();
                    }
                };
                new Thread(trasTarefa).start();
            }
        });
        contentPane.add(trasBtn);

        direitaBtn = new JButton("DIREITA");
        direitaBtn.setBounds(267, 84, 117, 29);
        direitaBtn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                Runnable direitaTarefa = new Runnable() {
                    @Override
                    public void run() {
                        direitaAction();
                    }
                };
                new Thread(direitaTarefa).start();
            }
        });
        contentPane.add(direitaBtn);

        esquerdaBtn = new JButton("ESQUERDA");
        esquerdaBtn.setBounds(45, 84, 117, 29);
        esquerdaBtn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                Runnable esquerdaTarefa = new Runnable() {
                    @Override
                    public void run() {
                        esquerdaAction();
                    }
                };
                new Thread(esquerdaTarefa).start();
            }
        });
        contentPane.add(esquerdaBtn);

        tipoRobotChkBox = new JCheckBox("Simulado/Real");
        tipoRobotChkBox.setBounds(10, 155, 115, 23);
        tipoRobotChkBox.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                if (tipoRobotChkBox.isSelected()) {
                    print("Modo Selecionado: Robot Simulado");
                    dados.getRobot().setRobot(new RobotSimulado());
```

```
272                 } else {
273                     print("Modo Selecionado: Robot Real");
274                     dados.getRobot().setRobot(new CRobotLegoEV3());
275                 }
276             }
277         });
278         contentPane.add(tipoRobotChkBox);
279 
280         gravadorPanel = new JPanel();
281         gravadorPanel.setOpaque(false);
282         gravadorPanel.setBorder(BorderFactory.createTitledBorder("Gravador"))
                ;
283         gravadorPanel.setBounds(6, 190, 455, 80);
284         gravadorPanel.setLayout(null);
285         contentPane.add(gravadorPanel);
286 
287         JLabel ficheiroLabel = new JLabel("Ficheiro");
288         ficheiroLabel.setBounds(10, 30, 50, 16);
289         gravadorPanel.add(ficheiroLabel);
290 
291         ficheiroField = new JTextField();
292         ficheiroField.setBounds(65, 25, 300, 26);
293         gravadorPanel.add(ficheiroField);
294         ficheiroField.setColumns(10);
295 
296         browseBtn = new JButton("...");
297         browseBtn.addActionListener(new ActionListener() {
298             public void actionPerformed(ActionEvent e) {
299                 JFileChooser fileChooser = new JFileChooser();
300                 int result = fileChooser.showOpenDialog(GUI_Gravador.this);
301                 if (result == JFileChooser.APPROVE_OPTION) {
302                     ficheiroField.setText(fileChooser.getSelectedFile().
                            getAbsolutePath());
303                     print("Ficheiro selecionado: " + ficheiroField.getText())
                            ;
304                 }
305             }
306         });
307         browseBtn.setBounds(370, 24, 45, 29);
308         gravadorPanel.add(browseBtn);
309 
310         gravarBtn = new JButton("Gravar");
311         gravarBtn.addActionListener(new ActionListener() {
312             public void actionPerformed(ActionEvent e) {
313                 if (!dados.isOnOff()) {
314                     print("Ligar o robot primeiro");
315                     return;
316                 }
317                 String caminho = ficheiroField.getText() != null ?
                        ficheiroField.getText().trim() : "";
318                 if (caminho.isEmpty()) {
```

```
                print("Selecione um ficheiro para gravar primeiro");
                return;
            }

            if (!dados.getGravador().isGravar()) {
                dados.getGravador().iniciarGravar(caminho);
                gravarBtn.setText("Parar");
            } else {
                dados.getGravador().pararGravacao();
                gravarBtn.setText("Gravar");
            }
        }
    });
    gravarBtn.setBounds(100, 50, 117, 29);
    gravadorPanel.add(gravarBtn);

    reproduzirBtn = new JButton("Reproduzir");
    reproduzirBtn.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            if (!dados.isOnOff()) {
                print("Ligar o robot");
            }

            String caminho = ficheiroField.getText() != null ?
                ficheiroField.getText().trim() : "";
            if (caminho.isEmpty()) {
                print("Selecione um ficheiro para reproduzir primeiro");
                return;
            }

            // se estiver a gravar, para primeiro
            if (dados.getGravador().isGravar()) {
                dados.getGravador().pararGravacao();
                gravarBtn.setText("Gravar");
            }

            if (!dados.getGravador().isReproduzir()) {
                dados.getGravador().reproduzir(caminho);
                reproduzirBtn.setText("Parar");
                print("Reproduzir");
            } else {
                dados.getGravador().pararReproducao();
                dados.getRobot().Parar(false);
                reproduzirBtn.setText("Reproduzir");
            }
        }
    });
    reproduzirBtn.setBounds(230, 50, 117, 29);
    gravadorPanel.add(reproduzirBtn);

    JLabel consolaLabel = new JLabel("Consola");
```

```
        consolaLabel.setBounds(6, 280, 61, 16);
        contentPane.add(consolaLabel);

        JScrollPane consolaScrollPane = new JScrollPane();
        consolaScrollPane.setBounds(6, 300, 455, 230);
        contentPane.add(consolaScrollPane);
        consolaTextArea = new JTextArea();
        DefaultCaret caret = (DefaultCaret) consolaTextArea.getCaret();
        caret.setUpdatePolicy(DefaultCaret.ALWAYS_UPDATE);
        consolaScrollPane.setViewportView(consolaTextArea);

        limparBtn = new JButton("Limpar");
        limparBtn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                consolaTextArea.setText(null); // Limpa a consola
            }
        });
        limparBtn.setBounds(120, 540, 117, 29);
        contentPane.add(limparBtn);

        imprimirChkBox = new JCheckBox("Imprimir");
        imprimirChkBox.setSelected(true);
        imprimirChkBox.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                if (imprimirChkBox.isSelected()) {
                    print("Impressão na consola ATIVADA.");
                }
            }
        });
        imprimirChkBox.setBounds(250, 542, 100, 23);
        contentPane.add(imprimirChkBox);
    }

    public void setDados(Dados dados) {
        this.dados = dados;
        // valores pre definidos gui
        raioField.setText(Integer.toString(dados.getRaio()));
        anguloField.setText(Integer.toString(dados.getAngulo()));
        distanciaField.setText(Integer.toString(dados.getDistancia()));
        robotField.setText(dados.getNomeRobot());

        nomeAtual = dados.getNomeRobot();
        if (dados.isOnOff()) {
            ligado = true;
            ligarChkBox.setSelected(true);
            print("Robot partilhado ativo: " + nomeAtual);
        }


    }

```

```
420     public void print(String s) {
421         if (imprimirChkBox == null) return;
422         Runnable runnable = () -> {
423             if (imprimirChkBox.isSelected()) {
424                 consolaTextArea.append(s + "\n");
425             }
426         };
427         if (SwingUtilities.isEventDispatchThread()) {
428             runnable.run();
429         } else {
430             SwingUtilities.invokeLater(runnable);
431         }
432     }
433 
434     private void frenteAction() {
435         if (!ligado) {
436             print("Robot não está ligado!");
437             return;
438         }
439 
440         try {
441             // TODO confirmar se é preciso tirar estas entradas de semáforo j
                    á que fazemos no MyRobotSync
442             MyRobotSync robot = getRobotAtual();
443             robot.entrar();
444             robot.Reta(dados.getDistancia());
445             dados.registarComandos(new Comando(Comando.ID_RETA, dados.
                    getDistancia()));
446             print("Comando enviado: Frente com distância " + dados.
                    getDistancia());
447             print("Espera: " + TempoMovimento.tempoReta(dados.getDistancia())
                    );
448             // TODO confirmar com stor
449             dados.registarComandos(new Comando(Comando.ID_PARAR, 0));
450             robot.Parar(false);
451         } catch (InterruptedException ex) {
452             Thread.currentThread().interrupt();
453         } finally {
454             getRobotAtual().sair();
455         }
456     }
457 
458     private void trasAction() {
459         if (!ligado) {
460             print("Robot não está ligado!");
461             return;
462         }
463         try {
464             MyRobotSync robot = getRobotAtual();
465             robot.entrar();
466             robot.Reta(dados.getDistancia() * -1);
```

```
467             dados.registarComandos(new Comando(Comando.ID_RETA, dados.
                    getDistancia() * -1));
468             print("Comando enviado: Trás com distância: " + dados.
                    getDistancia());
469             print("Espera: " + TempoMovimento.tempoReta(dados.getDistancia())
                    );
470             dados.registarComandos(new Comando(Comando.ID_PARAR, 0));
471             robot.Parar(false);
472         } catch (InterruptedException ex) {
473             Thread.currentThread().interrupt();
474         } finally {
475             getRobotAtual().sair();
476         }
477     }
478 
479     private void direitaAction() {
480         if (!ligado) {
481             print("Robot não está ligado!");
482             return;
483         }
484         try {
485             MyRobotSync robot = getRobotAtual();
486             robot.entrar();
487             robot.CurvarDireita(dados.getRaio(), dados.getAngulo());
488             dados.registarComandos(new Comando(Comando.ID_DIREITA, dados.
                    getRaio(), dados.getAngulo()));
489             print("Comando enviado: Curvar à Direita com ângulo " + dados.
                    getAngulo() + " e raio " + dados.getRaio());
490             print("Espera: " + TempoMovimento.tempoCurvar(dados.getRaio(),
                    dados.getAngulo()) + " ms");
491             dados.registarComandos(new Comando(Comando.ID_PARAR, 0));
492             robot.Parar(false);
493         } catch (InterruptedException ex) {
494             Thread.currentThread().interrupt();
495         } finally {
496             getRobotAtual().sair();
497         }
498     }
499 
500     private void esquerdaAction() {
501         if (!ligado) {
502             print("Robot não está ligado!");
503             return;
504         }
505         try {
506             MyRobotSync robot = getRobotAtual();
507             robot.entrar();
508             robot.CurvarEsquerda(dados.getRaio(), dados.getAngulo());
509             dados.registarComandos(new Comando(Comando.ID_ESQUERDA, dados.
                    getRaio(), dados.getAngulo()));
510             print("Comando enviado: Curvar à Esquerda com ângulo " + dados.
```

```
                getAngulo() + " e raio " + dados.getRaio());
            print("Espera: " + TempoMovimento.tempoCurvar(dados.getRaio(),
                dados.getAngulo()) + " ms");
            dados.registarComandos(new Comando(Comando.ID_PARAR, 0));
            robot.Parar(false);
        } catch (InterruptedException ex) {
            Thread.currentThread().interrupt();
        } finally {
            getRobotAtual().sair();
        }
    }

    public MyRobotSync getRobotAtual() {
        if (nomeAtual.equals(dados.getNomeRobot()) && dados.isOnOff()) {
            return dados.getRobot();
        }
        return robotLocal;
    }

    private boolean isPartilhado(){
        return nomeAtual.equals(dados.getNomeRobot());
    }

    public void resetBotaoReprod() {
        SwingUtilities.invokeLater(() -> reproduzirBtn.setText("Reproduzir"))
            ;
    }
}
```

Listing 9: *Classe GUIGravador*

Interface IRobotLegoEV3

```
public interface IRobotLegoEV3 {
    boolean OpenEV3(String s);
    void CloseEV3();
    void Reta(int d);
    void CurvarEsquerda(int raio, int angulo);
    void CurvarDireita(int raio, int angulo);
    void Parar(boolean p);
    int SensorToque(int toque);
}
```

Listing 10: *Código Interface IRobotLegoEV3*

Classe MovAleatorios

```
import java.util.Random;
```

```
2
3  public class MovAleatorios extends Tarefa {
4
5      private final Dados dados;
6      private final Random random = new Random();
7
8      public MovAleatorios(Dados dados) {
9          this.dados = dados;
10     }
11
12     @Override
13     void trabalharAtividade() throws InterruptedException {
14         dados.getGui().print("Movimentos Aleatorios: Iniciado.");
15         int num;
16         try {
17             num = (int) dados.getGui().getNumSpinner().getValue();
18         } catch (Exception e) {
19             num = 0;
20         }
21
22         // tratamento de erros
23         if (num <= 0) {
24             dados.getGui().print("Movimentos Aleatorios: número inválido (" +
                    num + ")");
25             estado = Estados.DORMIR;
26             return;
27         }
28
29         dados.getGui().print("Movimentos Aleatorios: novo lote de " + num + "
                comandos.");
30         boolean interrompido = false;
31
32         for (int i = 0; i < num && estado == Estados.TRABALHAR; i++) {
33             // verificação antes de cada comando
34             if (Thread.interrupted()) {
35                 dados.getGui().print("Movimentos Aleatorios: interrupção
                        antes do comando " + (i + 1));
36                 interrompido = true;
37                 estado = Estados.DORMIR;
38                 break;
39             }
40
41             int movimento = random.nextInt(3); // reta = 0, dir = 1, esq = 2
42             int distancia, raio, angulo;
43             long tempo;
44
45             switch (movimento) {
46                 case 0: // RETA
47                     distancia = randoms(10, 50);
48                     dados.getGui().print((i + 1) + ": Comando Aleatório: RETA
                            " + distancia + " cm");
```

```
49                  // exclusão mútua por comando
50                  dados.getRobot().entrar();
51                  try {
52                      dados.getRobot().Reta(distancia);
53                      dados.registarComandos(new Comando(Comando.ID_DIREITA
                            , distancia));
54                  } finally {
55                      dados.getRobot().sair();
56                  }
57                  tempo = TempoMovimento.tempoReta(distancia);
58                  break;
59
60              case 1: // DIREITA
61                  raio = randoms(10, 30);
62                  angulo = randoms(20, 90);
63                  dados.getGui().print((i + 1) + ": Comando Aleatório
                        DIREITA raio=" + raio + " ang=" + angulo);
64                  dados.getRobot().entrar();
65                  try {
66                      dados.getRobot().CurvarDireita(raio, angulo);
67                      dados.registarComandos(new Comando(Comando.ID_DIREITA
                            , raio, angulo));
68                  } finally {
69                      dados.getRobot().sair();
70                  }
71                  tempo = TempoMovimento.tempoCurvar(raio, angulo);
72                  break;
73
74              case 2: // ESQUERDA
75                  raio = randoms(10, 30);
76                  angulo = randoms(20, 90);
77                  dados.getGui().print((i + 1) + ": Comando Aleatório
                        ESQUERDA raio=" + raio + " ang=" + angulo);
78                  dados.getRobot().entrar();
79                  try {
80                      dados.getRobot().CurvarEsquerda(raio, angulo);
81                      dados.registarComandos(new Comando(Comando.
                            ID_ESQUERDA, raio, angulo));
82                  } finally {
83                      dados.getRobot().sair();
84                  }
85                  tempo = TempoMovimento.tempoCurvar(raio, angulo);
86                  break;
87
88              default:
89                  tempo = 0;
90          }
91
92          dados.getGui().print("Espera: " + tempo + " ms");
93          try {
94              Thread.sleep(tempo);
```

```
            } catch (InterruptedException e) {
                dados.getGui().print("Movimentos Aleatorios: comando
                    interrompido durante espera");
                interrompido = true;
                break;
            }
        }

        if (!interrompido) {
        //dados.getRobot().Parar(false);
        dados.getGui().print("Movimentos Aleatorios: lote concluído.");
        dados.getRobot().Parar(false);
        Thread.sleep(3000);

        } else {
            dados.getGui().print("Movimentos Aleatorios: lote interrompido");
        }
        if (interrompido) {
            Thread.interrupted();
        }
        dados.getGui().print("Movimentos Aleatorios: desativado.");

    }
    private int randoms(int min, int max) {
        return random.nextInt((max - min) + 1) + min;
    }
}
```

Listing 11: *Código Classe MovAleatorios*

Classe MyRobotLegoEV3

```
import java.util.concurrent.Semaphore;

/**
 * Classe servidor do RobotLegoEV3
 * CONSUMIDOR!!
 */
public class MyRobotLegoEV3 extends Tarefa implements IRobotLegoEV3 {

    private static double VEL_ROBOT = 0.02; // read only, fundamental para
        esperar o tempo
    private static int TEMPO_COMUNICACAO = 100;
    private BufferCircularComando buffer;
    private IRobotLegoEV3 robot;
    private volatile boolean stopRequested;
    private Semaphore semaphore;


    public MyRobotLegoEV3() {
```

```
18          super();
19          robot = new RobotSimulado();
20          buffer = new BufferCircularComando();
21          stopRequested = false;
22
23      }
24
25      @Override
26      void trabalharAtividade() {
27          Comando comando = buffer.removerElemento();
28
29          if (comando == null) return;
30
31          stopRequested = false;
32          long tempoEspera = 0;
33
34          switch (comando.getIdentificador()) {
35              case Comando.ID_RETA:
36                  int d = comando.getArg1();
37                  robot.Reta(d);
38                  tempoEspera = TempoMovimento.tempoReta(d);
39                  break;
40              case Comando.ID_DIREITA:
41                  int raioD = comando.getArg1();
42                  int anguloD = comando.getArg2();
43                  robot.CurvarDireita(raioD, anguloD);
44                  tempoEspera = TempoMovimento.tempoCurvar(raioD, anguloD);
45                  break;
46              case Comando.ID_ESQUERDA:
47                  int raioE = comando.getArg1();
48                  int anguloE = comando.getArg2();
49                  robot.CurvarEsquerda(raioE, anguloE);
50                  tempoEspera = TempoMovimento.tempoCurvar(raioE, anguloE);
51                  break;
52              case Comando.ID_PARAR:
53                  robot.Parar(true);
54                  tempoEspera = TEMPO_COMUNICACAO;
55                  break;
56
57          }
58
59          if (tempoEspera > 0) {
60              espera(tempoEspera);
61          }
62      }
63
64      private void espera(long tempoTotal) {
65          long tempoDecorrido = 0;
66          int parte = 10; // verificação a cada 10ms
67
68          while (tempoDecorrido < tempoTotal) {
```

```
            if (stopRequested || estado != Estados.TRABALHAR) break;
            try {
                Thread.sleep(parte);
            } catch (InterruptedException e) {
                break;
            }
            tempoDecorrido += parte;
        }

    }

    public void Reta(int distancia) {
        Comando c = new Comando(Comando.ID_RETA, distancia);
        buffer.inserirElemento(c);
    }

    public void CurvarDireita(int raio, int angulo) {
        Comando c = new Comando(Comando.ID_DIREITA, raio, angulo);
        buffer.inserirElemento(c);
    }

    public void CurvarEsquerda(int raio, int angulo) {
        Comando c = new Comando(Comando.ID_ESQUERDA, raio, angulo);
        buffer.inserirElemento(c);

    }

    public void Parar(boolean parar) {
        if (parar) {
            stopRequested = true;
            buffer.limparBuffer();
            robot.Parar(true);
        } else {
            Comando c = new Comando(Comando.ID_PARAR, 0);
            buffer.inserirElemento(c);
        }
    }

    @Override
    public int SensorToque(int toque) {
        return robot.SensorToque(toque);
    }

    public boolean OpenEV3(String s) {
        return robot.OpenEV3(s);
    }

    public void CloseEV3() {
        robot.CloseEV3();
    }

```

```
120     public void setSemaphore(Semaphore s) {
121         this.semaphore = s;
122     }
123 
124     /**
125      * Ler do comando, enviar para o robot, e esperar pelo tempo de execução
             de forma
126      * interrompível para permitir PARAR imediato.
127      */
128     /*public void run() {
129         while (!Thread.currentThread().isInterrupted()) {
130             Comando comando = buffer.removerElemento();
131             if (comando == null) {
132                 pararImediatamente();
133                 continue;
134             }
135             try {
136                 if (semaphore != null) {
137                     semaphore.acquire();
138                 }
139                 long t;
140                 try {
141                     t = comando.executarComandos(robot);
142                 } finally {
143                     if (semaphore != null) semaphore.release();
144                 }
145                 if (t > 0) Thread.sleep(t);
146             } catch (InterruptedException e) {
147                 pararImediatamente();
148             }
149         }
150         try {
151             if (semaphore != null) semaphore.acquire();
152             robot.Parar(true);
153         } catch (InterruptedException e) {
154 
155         } finally {
156             if (semaphore != null) semaphore.release();
157         }
158     }
159 */
160     public IRobotLegoEV3 getRobot() {
161         return robot;
162     }
163 
164     public void setRobot(IRobotLegoEV3 robot) {
165         this.robot = robot;
166     }
167 }
```

Listing 12: *Código Classe MyRobotLegoEV3*

Classe MyRobotSync

```
import java.util.concurrent.Semaphore;

public class MyRobotSync implements IRobotLegoEV3 {
    private Semaphore acesso;
    private MyRobotLegoEV3 robot;

    public MyRobotSync() {
        acesso = new Semaphore(1);
        robot = new MyRobotLegoEV3();
        robot.setSemaphore(acesso);
    }

    @Override
    public boolean OpenEV3(String s) {
        return robot.OpenEV3(s);
    }

    @Override
    public void CloseEV3() {
        robot.CloseEV3();
    }

    @Override
    public void Reta(int d) {
        robot.Reta(d);
    }

    @Override
    public void CurvarEsquerda(int raio, int angulo) {
        robot.CurvarEsquerda(raio, angulo);
    }

    @Override
    public void CurvarDireita(int raio, int angulo) {
        robot.CurvarDireita(raio, angulo);
    }

    @Override
    public void Parar(boolean p) {
        robot.Parar(p);
    }

    @Override
    public int SensorToque(int toque) {
        return robot.SensorToque(toque);
    }

    public IRobotLegoEV3 getRobot() {
        return robot;
```

```
    }

    public void setRobot(IRobotLegoEV3 robot) {
        this.robot.setRobot(robot);
    }

    public void entrar() throws InterruptedException {
        acesso.acquire();
    }

    public void sair() {
        acesso.release();
    }

    public void start() {
        robot.start();
    }

    public void stop() {
        robot.interrupt();
    }
}
```

Listing 13: *Código Classe MyRobotSync*

Classe RobotSimulado

```
public class RobotSimulado implements IRobotLegoEV3 {
    public RobotSimulado() {
    }
    @Override
    public boolean OpenEV3(String s) {
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
        }
        return true;
    }

    @Override
    public void CloseEV3() {
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
        }
    }

    @Override
    public void Reta(int d) {
        try {
```

```
            long t = TempoMovimento.tempoReta(d);
            Thread.sleep(t);
        } catch (InterruptedException e) {
        }
    }

    @Override
    public void CurvarEsquerda(int raio, int angulo) {
        try {
            long t = TempoMovimento.tempoCurvar(raio, angulo);
            Thread.sleep(t);
        } catch (InterruptedException e) {
        }
    }

    @Override
    public void CurvarDireita(int raio, int angulo) {
        try {
            long t = TempoMovimento.tempoCurvar(raio, angulo);
            Thread.sleep(t);
        } catch (InterruptedException e) {
        }
    }

    @Override
    public void Parar(boolean p) {
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
        }
    }

    @Override
    public int SensorToque(int toque) {
        return 0;
    }
}
```

Listing 14: *Código Classe RobotSimulado*

Classe Abstrata Tarefa

```
import java.util.concurrent.Semaphore;
public abstract class Tarefa extends Thread {

    public Semaphore semaforo_dormir;
    public Estados estado;

    public Tarefa() {
        estado = Estados.DORMIR;
```

```
9           semaforo_dormir = new Semaphore(0);
10      }
11
12      /**
13       * Este trabalhar é geral. Todas as tarefas que estendem desta vão
             obrigatoriamente
14       * ter este método. Mas como as tarefas vão ser diferentes a implementaçã
             o do método vai mudar
15       */
16      abstract void trabalharAtividade() throws InterruptedException;
17
18      public void dormir() throws InterruptedException {
19          semaforo_dormir.acquire();
20      }
21
22      public void ativar() {
23          estado = Estados.TRABALHAR;
24          semaforo_dormir.release();
25      }
26
27      public void desativar() {
28          estado = Estados.DORMIR;
29      }
30
31      public void run() {
32          while (estado != Estados.TERMINAR) {
33              switch (estado) {
34                  case DORMIR:
35                      try {
36                          dormir();
37                      } catch (InterruptedException e) {
38                          e.printStackTrace();
39                      }
40                      break;
41                  case TRABALHAR:
42                      try {
43                          trabalharAtividade();
44                      } catch (InterruptedException e){
45                          e.printStackTrace();
46                      }
47                      break;
48                  case TERMINAR:
49                      desativar();
50                      break;
51              }
52          }
53      }
54      public enum Estados {
55          TERMINAR,
56          DORMIR,
57          TRABALHAR
```

```
    }
}
```

Listing 15: *Código Classe Abstrata Tarefa*

Classe Auxiliar TempoMovimento

```
public final class TempoMovimento {
    private static double VEL_ROBOT_CM_POR_MS = 0.02;
    private static int TEMPO_COMUNICACAO_MS = 100;

    private TempoMovimento() {}

    public static long tempoReta(int distanciaCm) {
        return (long) (Math.abs(distanciaCm) / VEL_ROBOT_CM_POR_MS +
            TEMPO_COMUNICACAO_MS);
    }

    public static long tempoCurvar(int raioCm, int anguloGraus) {
        double comprimentoArco = raioCm * Math.toRadians(anguloGraus);
        return (long) (comprimentoArco / VEL_ROBOT_CM_POR_MS +
            TEMPO_COMUNICACAO_MS);
    }

    public static double getVelRobot() { return VEL_ROBOT_CM_POR_MS; }
    public static void setVelRobot(double velRobotCmPorMs) {
        VEL_ROBOT_CM_POR_MS = velRobotCmPorMs; }

    public static int getTempoComunicacao() { return TEMPO_COMUNICACAO_MS; }
    public static void setTempoComunicacao(int tempoMs) {
        TEMPO_COMUNICACAO_MS = tempoMs; }
}
```

Listing 16: *Código Classe Auxiliar TempoMovimento*